Agents in action

Francesco Gianferrari Pini — Quantyca


Capire l'LLM


Conversare


Scegliere il modello


Agire


Estendere l'agente


Programmare


Progettare il contesto

SEZIONE 1 DI 7

Capire l'LLM

Il motore dentro ogni agente



Capire l'LLM



Conversare



Scegliere il modello



Agire



Estendere l'agente



Programmare

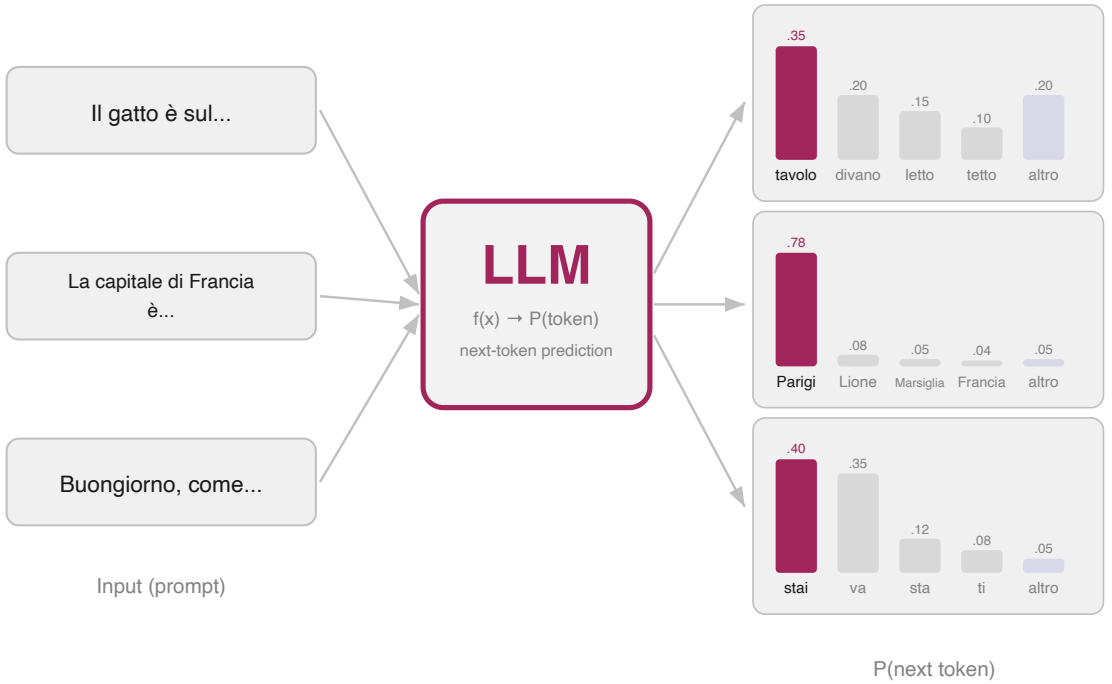


Progettare il contesto

LLM: il cuore degli agenti

> Un LLM è una funzione matematica che, data una sequenza di token, produce una distribuzione di probabilità sul prossimo token.

- **Input**: una sequenza di token — le unità in cui l'LLM "vede" il testo, approssimativamente parole o frammenti di parole.
- **Output**: una distribuzione di probabilità che assegna un valore a ogni token del vocabolario (decine di migliaia di possibilità).
- **Nessun concetto di "conversazione"**: a questo livello l'LLM non sa di parlare con qualcuno. È una funzione che completa testo plausibile.



Come si addestra un LLM



ripetuto trilioni di volte

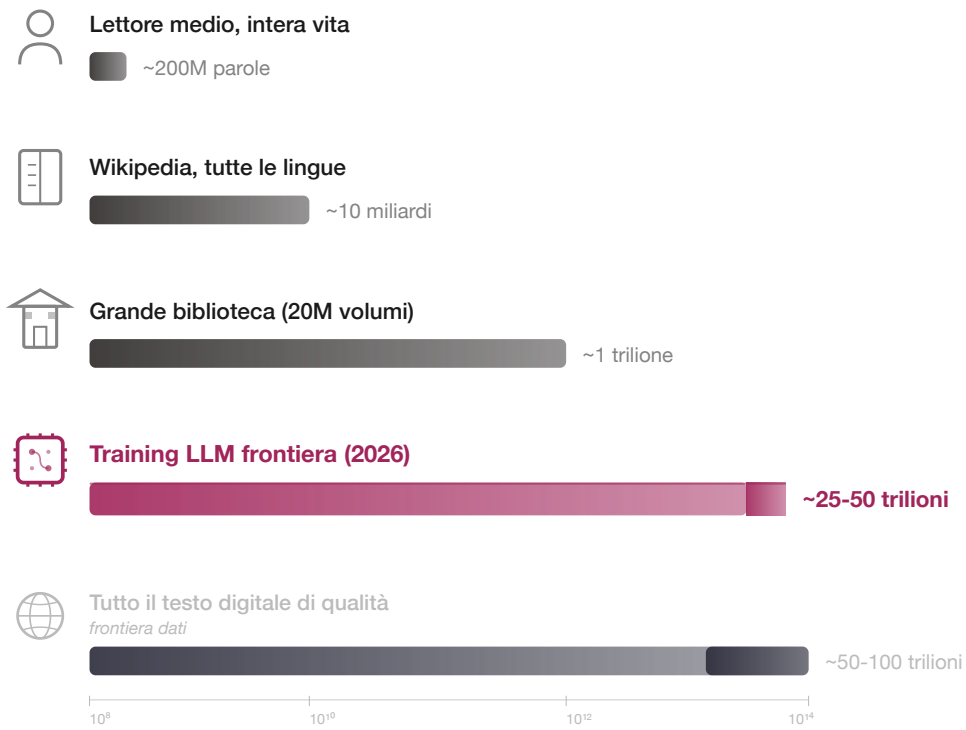


Miliardi di parametri partono casuali. Trilioni di micro-aggiustamenti li portano a predire il prossimo token in modo sempre più accurato.

Il pre-training di un LLM

Come nasce questa funzione

- **Pre-training**: addestramento su enormi corpus di testo (web, libri, codice).
- **Obiettivo**: prevedere il prossimo token a partire dai precedenti.
- **Scala tipica (2026)**: 25–50 trilioni di token in input, 400 miliardi – 1 trilione+ di parametri.



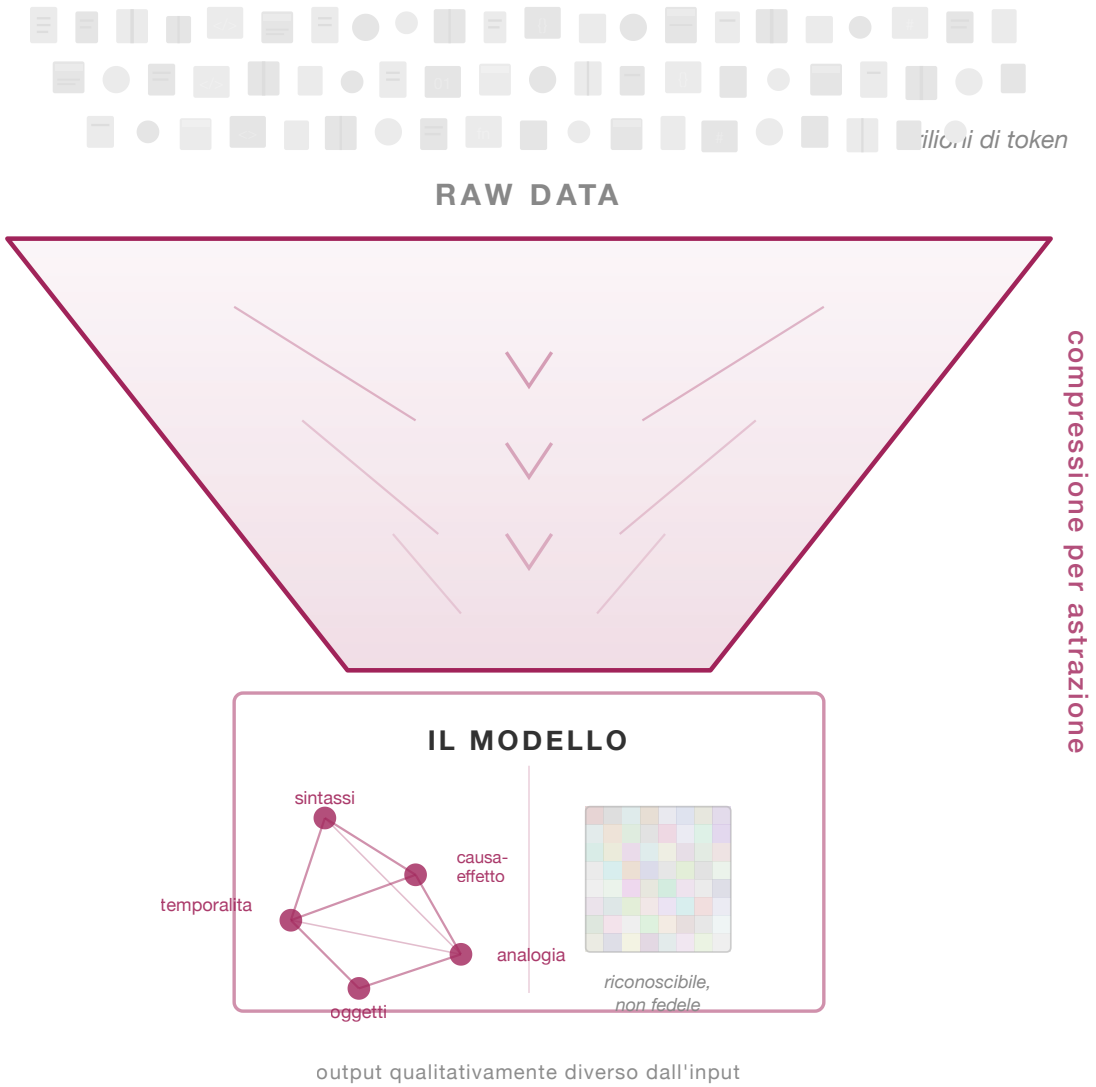
> *Alla fine del training, il modello è in grado di completare ogni frase attingendo ad ogni elemento di conoscenza, ma non necessariamente è "utile" a sostenere una conversazione o risolvere un problema.*

LLM: Compression through Abstraction

> **Per far stare trilioni di token in miliardi di parametri, il modello è costretto a costruire rappresentazioni astratte — non memorizza, modella.**

- Non c'è spazio per memorizzare letteralmente il training set.
- Il modello "impara" concetti, relazioni, regolarità del mondo.
- Le capacità emergenti (ragionamento, analogia, transfer) sono sottoprodotto della compressione, non funzionalità programmate.

> *"ChatGPT è un JPEG sfocato del web, che, per comprimere, ha dovuto capire."*
>
> — Ted Chiang, The New Yorker



La portata della conoscenza

Una conseguenza del training su scala

Ampiezza

L'LLM ha "letto" ordini di grandezza più testo di qualunque singolo essere umano.

→ Implicazione

Su qualsiasi dominio di conoscenza pubblica (medicina, diritto, codice, marketing, ingegneria) la baseline informativa è superiore a quella di un laureato medio nel campo.

Indipendentemente dall'ambito di utilizzo, un modello moderno sfrutta una mole enorme di informazioni (non sempre di altissima qualità o veridicità) per dare una risposta. È certamente già una capacità "super-umana", e nasce dalla capacità di comprimere attraverso una astrazione.

Conseguenze della compressione

✓ Dove è forte

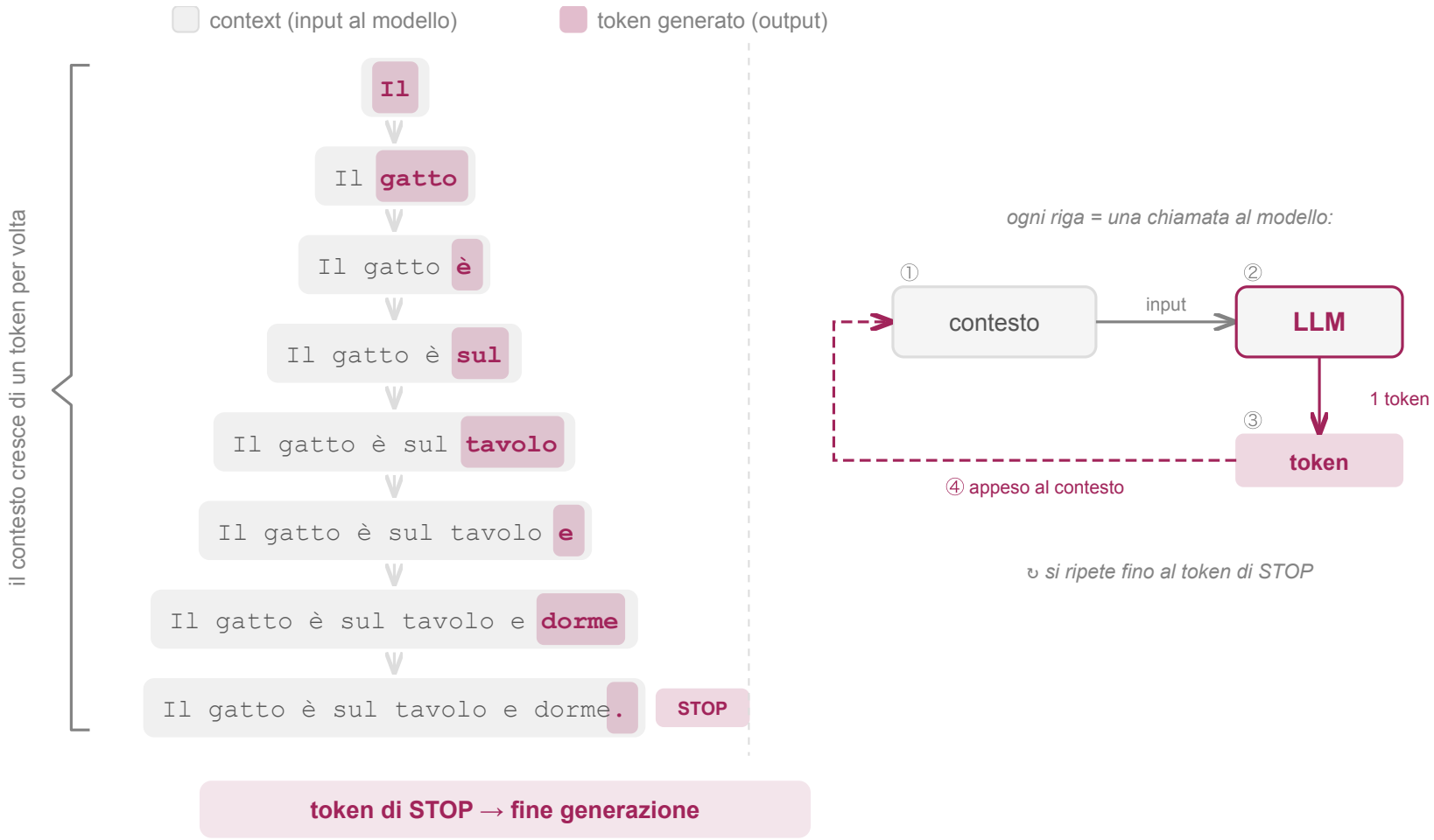
- Framing concettuali su qualunque dominio
- Ragionamento e analogia
- Trasformazione di testo (traduzione, sintesi, riformattazione)

⚠ Dove serve cautela

- Fatti puntuali (date, numeri, citazioni letterali)
- Conoscenza post-cutoff
- Dettagli tecnici su nicchie poco rappresentate
- Dati proprietari non nel training
- Calcoli numerici precisi e passaggi lunghi

Fidati del framing, verifica i fatti. Le allucinazioni non sono un bug morale — sono il comportamento strutturale di un compressore lossy a cui chiedi di ricostruire ciò che non ha memorizzato bene.

Loop 1/4: Come l'LLM genera testo



Il modello non pianifica una risposta. Produce un token alla volta. Ogni nuovo token viene appeso al contesto e il contesto completo viene rivalutato per produrre il successivo.

Cosa significa in pratica?

- **Stesso prompt ≠ stessa risposta**: il sampling è probabilistico. Il non-determinismo è strutturale, non un bug.
- **Lo streaming non è UX**: le parole che compaiono una per una riflettono la meccanica reale. Il modello genera mentre l'utente legge.
- **Non c'è pianificazione globale**: il modello non "sa dove sta andando" prima di iniziare. Ogni token è locale.

> Questo è il primo mattoncino. I prossimi tre loop — conversazione, tool, codice — lo avvolgono uno sull'altro.


MEMENTO
GUY PEARCE CARRIE-ANNE MOSS JOE PANTOLIANO
MEMENTO

Il limite della context window

Il protagonista di Memento. Un uomo colpito da amnesia anterograda — ricorda la sua vita prima dell'incidente, ma ogni nuovo ricordo svanisce in pochi minuti. Sopravvive scrivendo tutto su Polaroid e tatuaggi: tutto ciò che gli serve sapere ora deve essere fisicamente davanti ai suoi occhi, ora.

Un LLM funziona allo stesso modo. Il training è la "vita prima": conoscenza enorme, ma congelata. Tutto il resto — la conversazione in corso, i documenti caricati, i risultati dei tool — esiste per il modello solo se è dentro la finestra di contesto in questo preciso istante. Niente viene automaticamente ricordato tra una chiamata e l'altra.

Costo

Il meccanismo di attention è $O(n^2)$ sulla lunghezza del contesto. Raddoppiare il contesto più che raddoppia il costo di calcolo.

Qualità

Anche quando "ci sta", il modello non usa bene tutto il contesto. Tende a ricordare meglio inizio e fine, peggio il centro — lost in the middle.

SEZIONE 2 DI 7

Conversare

Come un modello impara a dialogare


Capire l'LLM


Conversare


Scegliere il modello


Agire


Estendere l'agente


Programmare


Progettare il contesto

Il primo agente: la chat

Dalla next token prediction alla conversazione

Prima — modello base

Completa testo plausibile. Data una sequenza, produce la continuazione statisticamente più probabile. Non sa di parlare con qualcuno, non distingue una domanda da un frammento di articolo, non segue istruzioni.

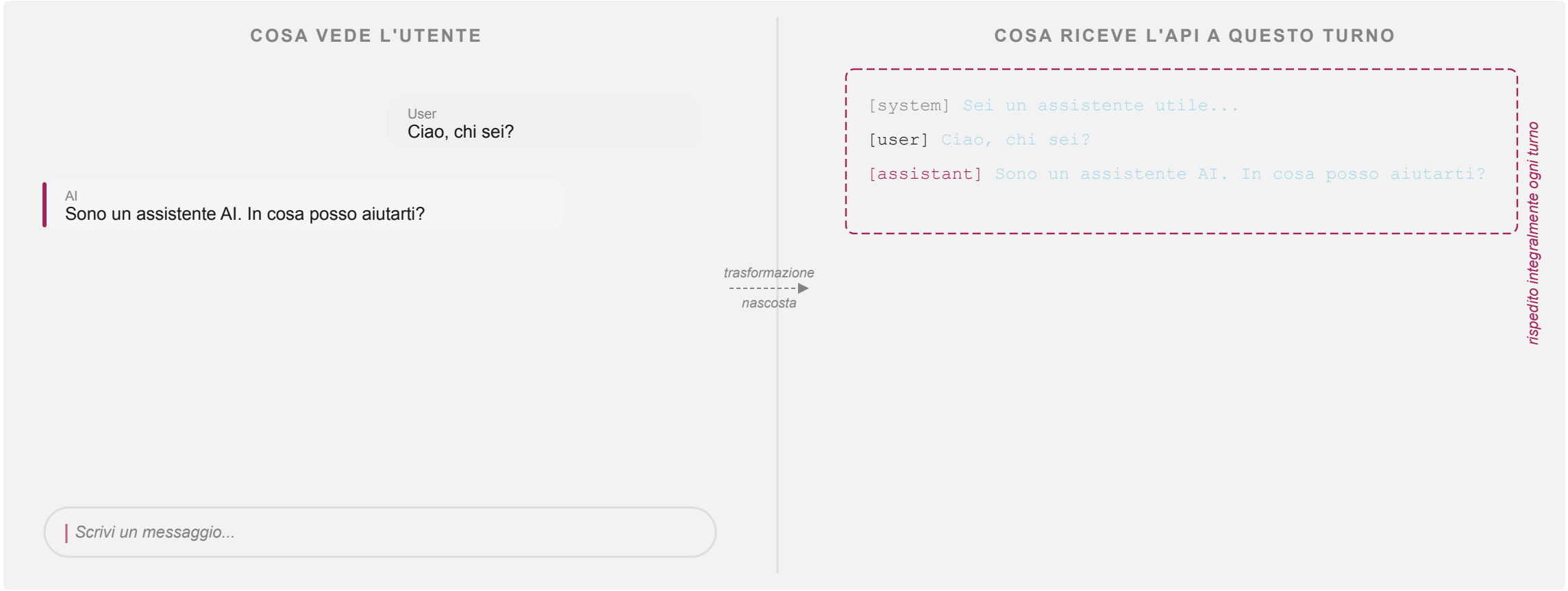
Dopo — agente conversazionale

Risponde a un interlocutore, segue istruzioni, mantiene un ruolo, rispetta vincoli di tono e comportamento.

Come si ottiene: un secondo giro di addestramento sopra il pre-training — instruction tuning e RLHF — che insegna al modello non cosa è probabile, ma cosa è utile rispondere. Più un'interfaccia che struttura lo scambio in turni.

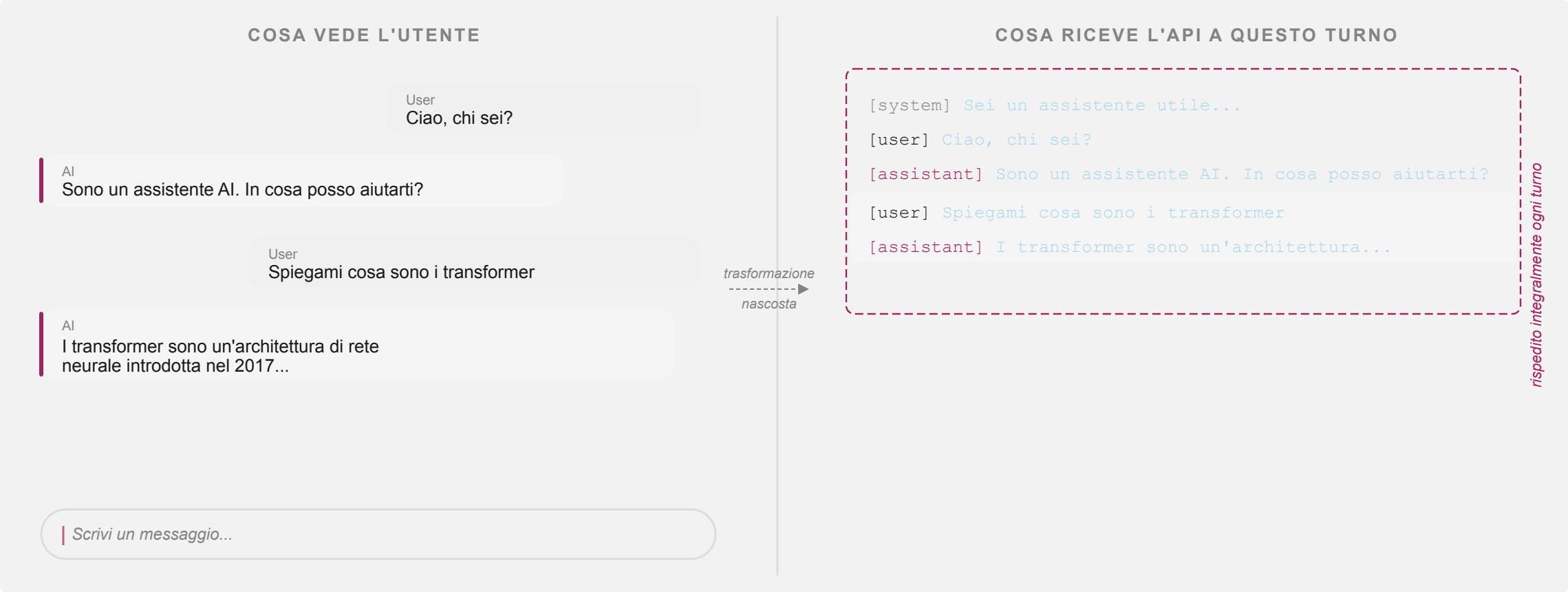
Novembre 2022, ChatGPT. Non un nuovo modello — GPT-3 esisteva dal 2020. Un nuovo modo di addestrarlo a conversare.

La meccanica della conversazione



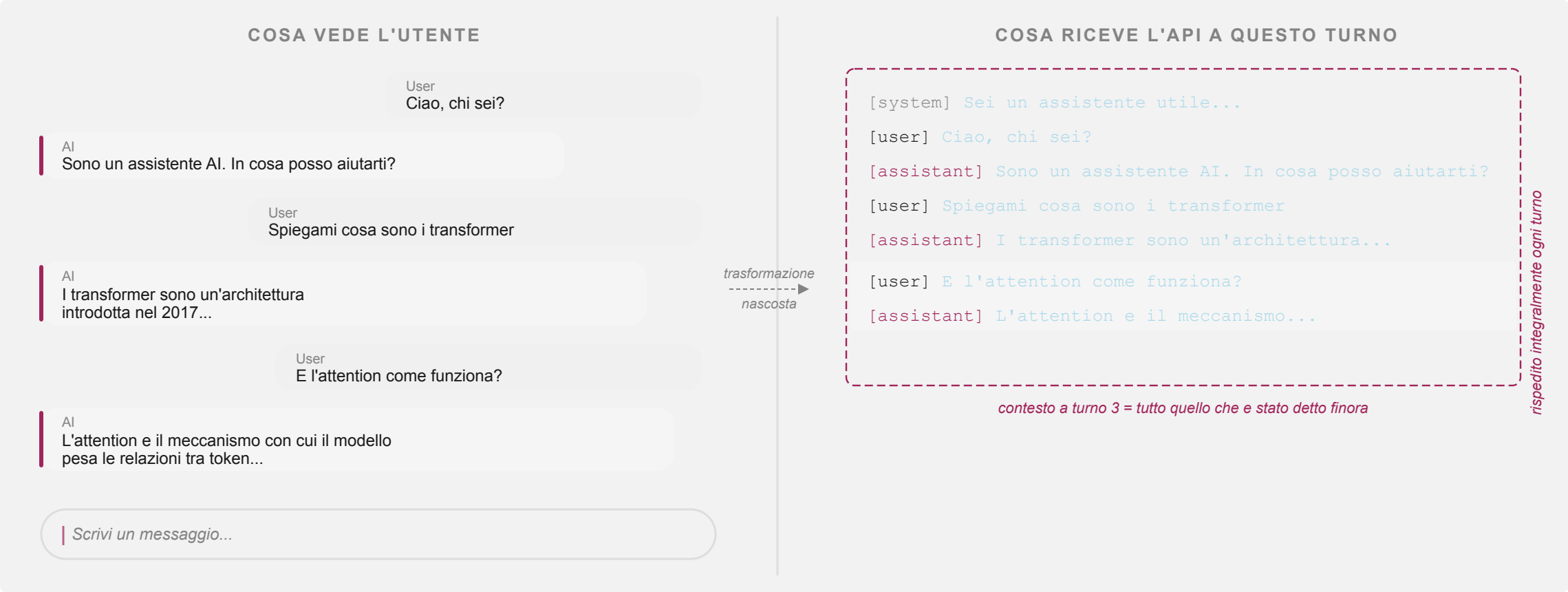
La conversazione è un artefatto dell'interfaccia. Per il modello è un unico testo strutturato con tag di ruolo, rispedito integralmente ad ogni turno. L'API è stateless — nessuna memoria tra chiamate.

La meccanica della conversazione



La conversazione è un artefatto dell'interfaccia. Per il modello è un unico testo strutturato con tag di ruolo, rispedito integralmente ad ogni turno. L'API è stateless — nessuna memoria tra chiamate.

La meccanica della conversazione



La conversazione è un artefatto dell'interfaccia. Per il modello è un unico testo strutturato con tag di ruolo, rispedito integralmente ad ogni turno. L'API è stateless — nessuna memoria tra chiamate.

Perché serve un secondo addestramento

Prompt dell'utente

Come posso aumentare le vendite?

B Modello base (solo pre-training)

```
Come posso aumentare le vendite?
E quante persone lavorano nella tua
azienda? Il settore è B2B o B2C? In quale
mercato geografico operate? Negli ultimi
anni la domanda è cresciuta o...
```

Il modello continua la frase come un testo sul web — con altre domande, non con una risposta.

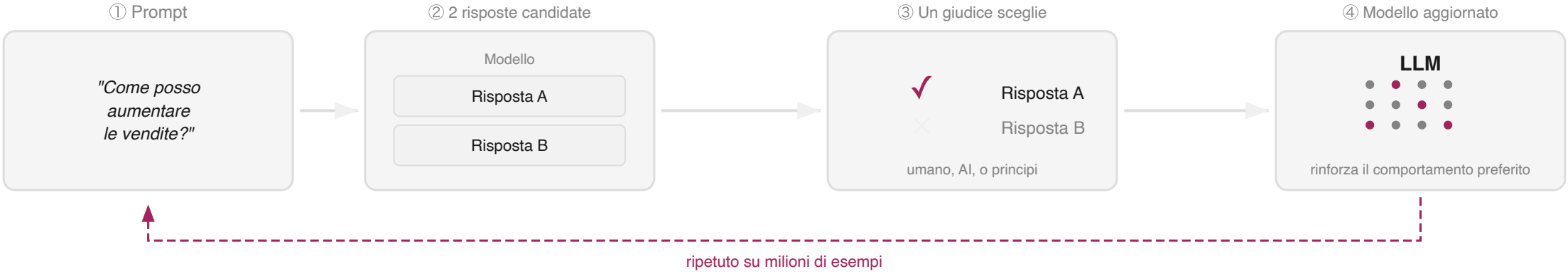
R Modello RLHF

Per aumentare le vendite puoi agire su tre leve: 1) acquisire nuovi clienti (marketing, partnership); 2) aumentare il valore medio per cliente (upselling, bundle); 3) migliorare la retention (CRM, customer success). Vuoi approfondire una di queste?

Il modello riconosce una domanda e fornisce una risposta utile, strutturata, orientata all'azione.

Da «cosa è probabile» a «cosa serve rispondere».

Il secondo addestramento: RLHF



Varianti: RLHF (feedback umano), RLAIF (feedback da AI), Constitutional AI (Anthropic), DPO. Il principio — premiare le risposte desiderabili, penalizzare le altre — è lo stesso.

- La personalità dei diversi modelli (Claude, GPT, Gemini) non è emergente — è **scolpita in post-training** con valori diversi.
- I limiti di comportamento non sono quasi mai limiti di capacità — sono **scelte di policy** del provider.
- Chi fornisce feedback plasma l'AI. Implicazioni etiche, geopolitiche, di business.

> Il modello che si usa non è "l'AI". È una particolare AI addestrata con particolari valori da particolari persone.

Il turno zero: il system prompt

> **Turno zero invisibile, messo in cima al contesto dal provider o dallo sviluppatore. Definisce ruolo, tono, vincoli e capacità dell'assistente.**

- Il modello tende a seguirlo ma non è obbligato — può essere eroso in conversazioni lunghe, sovrascritto da istruzioni forti, o aggirato (prompt injection).
- Ogni applicazione che usa LLM ha un system prompt. ChatGPT, Claude.ai, Gemini sono system prompt diversi oltre a modelli diversi.
- La personalità di un prodotto AI vive in gran parte qui.

> Il modello è uguale per tutti. Il system prompt è l'elemento principale di indirizzo e personalizzazione

Mappa dei task: forze e fragilità



Da qui in poi, ogni tecnica che vedremo (tool, RAG, codice, sub-agent) è un modo di rinforzare un task dove il modello da solo è fragile.

Il valore delle conversazioni

Traiettoria — una conversazione completa vista come sequenza di mosse: prompt utente → risposta del modello → riformulazione → correzione → approvazione. Ogni conversazione lascia una traccia ricca di segnali su cosa funziona e cosa no.

Per l'utente

- Ogni conversazione è un mezzo per risolvere un problema oggi
- Stateless: dimenticata domani dal sistema

Per il provider

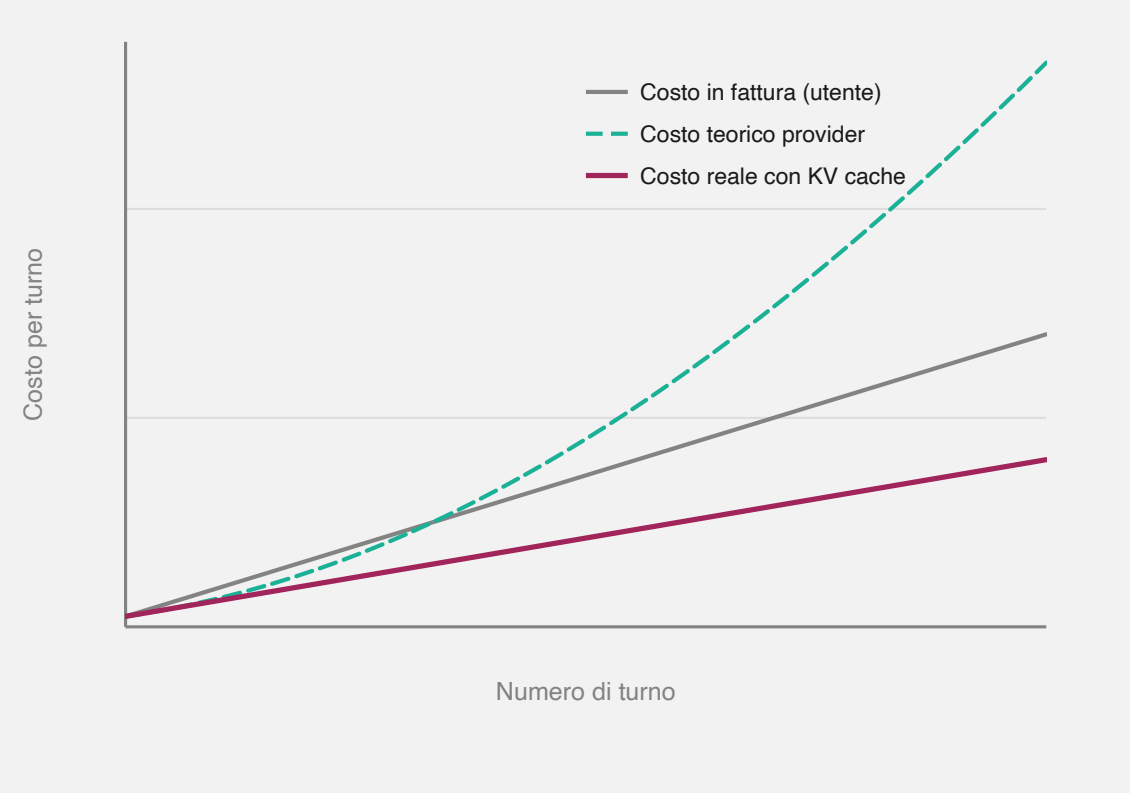
- Ogni conversazione è una traiettoria con prompt reale, risposte, correzioni, feedback
- Miliardi di traiettorie = carburante del prossimo training
- Emergono pattern d'uso che nessun test artificiale produce

Il moat non è il modello — il modello diventa commodity. Il moat sono le traiettorie di uso reale, che nessuno può replicare.

L'API è stateless: cosa significa davvero

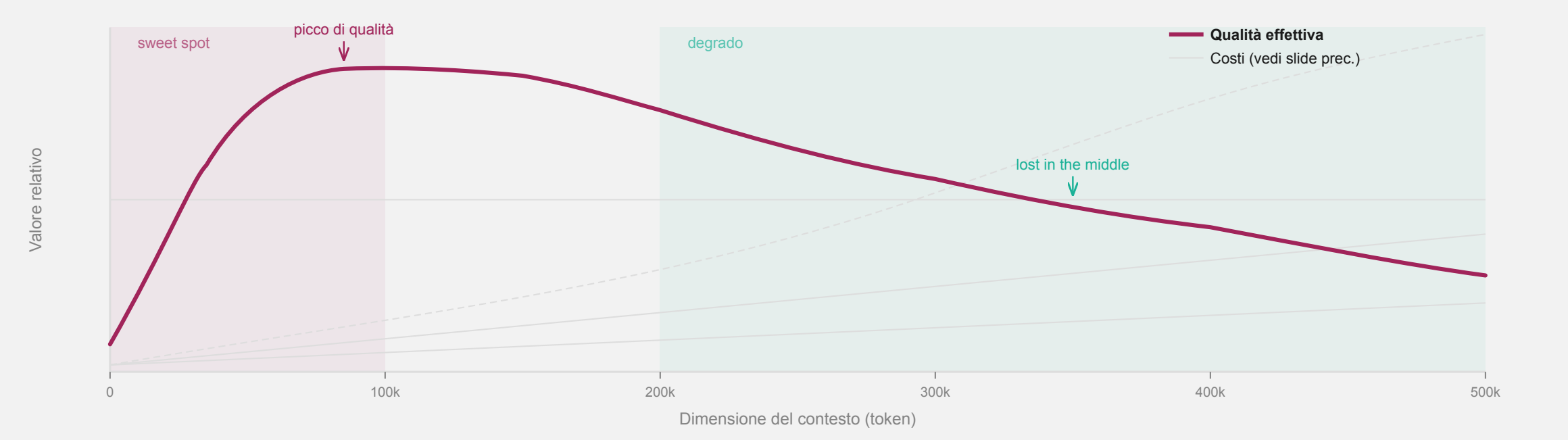
- L'API non conserva stato tra chiamate.
- Ad ogni nuovo turno, il contesto completo (system + storico + nuovo input) viene inviato da capo al modello.
- Il contesto cresce linearmente con la conversazione.

KV cache: trucco ingegneristico che memorizza le rappresentazioni (key/value) dei token già processati. Ad ogni turno il provider riusa il lavoro già fatto e processa solo i token nuovi. Senza KV cache, i costi crescerebbero quadraticamente.



Il costo per turno e la qualità della risposta dipendono dalla dimensione cumulativa del contesto, non dalla singola domanda.

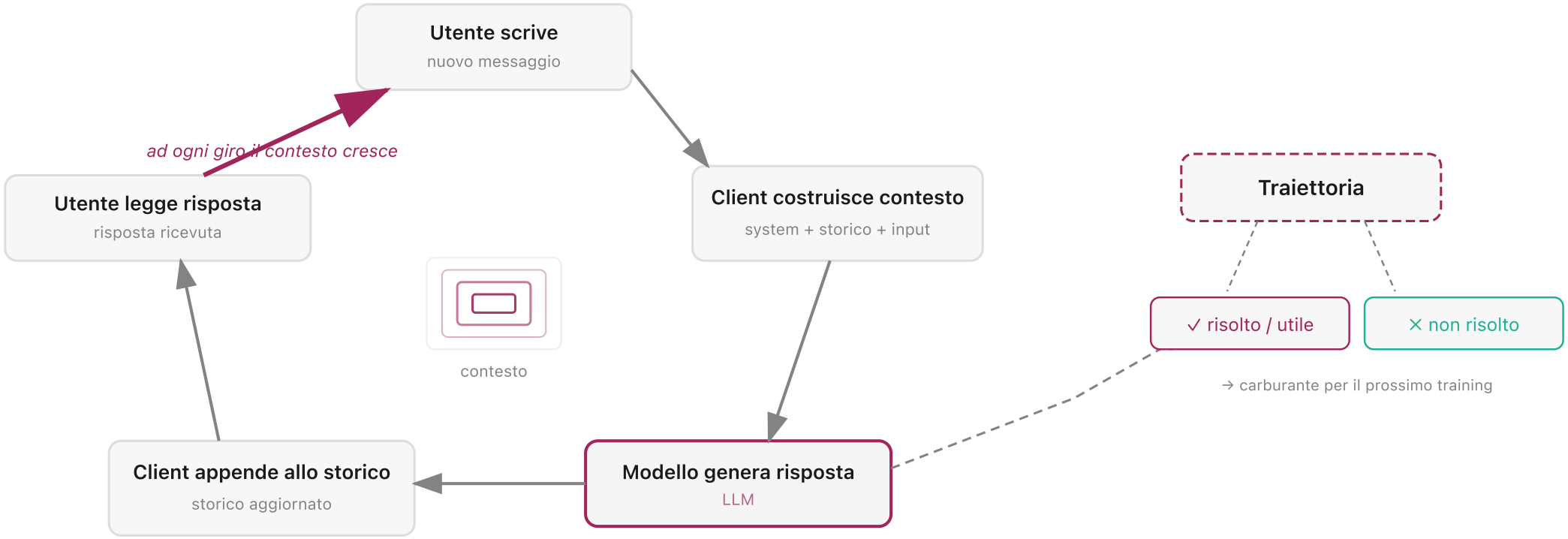
Più contesto ≠ più qualità



Costo cresce, latenza cresce, qualità cala. Il contesto non è mai gratis.

- **Nuovo compito, nuova chat.** Non "continuare" per efficienza — stai peggiorando, non migliorando.
- **Il contesto è una risorsa scarsa**, non un cassetto infinito.
- Sub-agent, context engineering, compaction, memoria: sono tutti risposte a questo problema.

Loop 2/4: la conversazione



Il loop turno-per-turno. Ad ogni giro il contesto cresce. Il problema da risolvere è la gestione di questa crescita.

SEZIONE 3 DI 7

Scegliere il modello

Closed vs Open: due strade


Capire l'LLM


Conversare


Scegliere il modello


Agire


Estendere l'agente


Programmare


Progettare il contesto

Closed Model vs Open Weights vs Open Source

Categoria	Cosa è aperto	Esempi
Closed / API-only	Niente. Accesso solo via API del provider	GPT, Claude, Gemini
Open weight	Pesi scaricabili. Training code e dataset tipicamente no	Llama (Meta), Mistral, DeepSeek, Qwen, Gemma
Open source "vero"	Pesi + training code + dataset	OLMo (AllenAI) — tipicamente dietro la frontiera

> "Open" quasi sempre significa open weight. I pesi sono scaricabili, ma training data e codice restano chiusi.

Trade-off decisionale: quando closed, quando open

Dimensione	Closed	Open
Qualità di frontiera	In testa oggi	A 6-12 mesi, ma il gap si chiude
Costo a scala	Pay-per-token, lineare con l'uso	Infrastruttura fissa, marginale ~zero
Privacy / data residency	Dati al provider (tranne enterprise)	Totalmente in casa, sovrano
Customization	Fine-tuning limitato, no accesso ai pesi	Fine-tuning completo, quantizzazione, RAG
Effort di adozione	Zero infrastruttura	Richiede GPU, MLOps, competenze
Geopolitica / compliance	Dipendenza da vendor USA o Cina	Sovranità, compatibilità AI Act europeo

> Con closed: paghi il token **e** regali le traiettorie. Con open: paghi l'infrastruttura, le traiettorie restano tue.

AGGIORNAMENTO 2026

Modelli open-weight da ~30B parametri (es. Gemma 4) raggiungono qualità comparabile a closed di classe Sonnet 4.5 su molti task, eseguibili su singola GPU consumer. Il ritardo rispetto alla frontiera rimane reale: ~6 mesi.

Closed per sperimentare velocemente, open per scalare volumi, privacy, customization. La scelta è per use case, non ideologica.

SEZIONE 4 DI 7

Agire

Dall'agente che parla a quello che fa



Un esempio

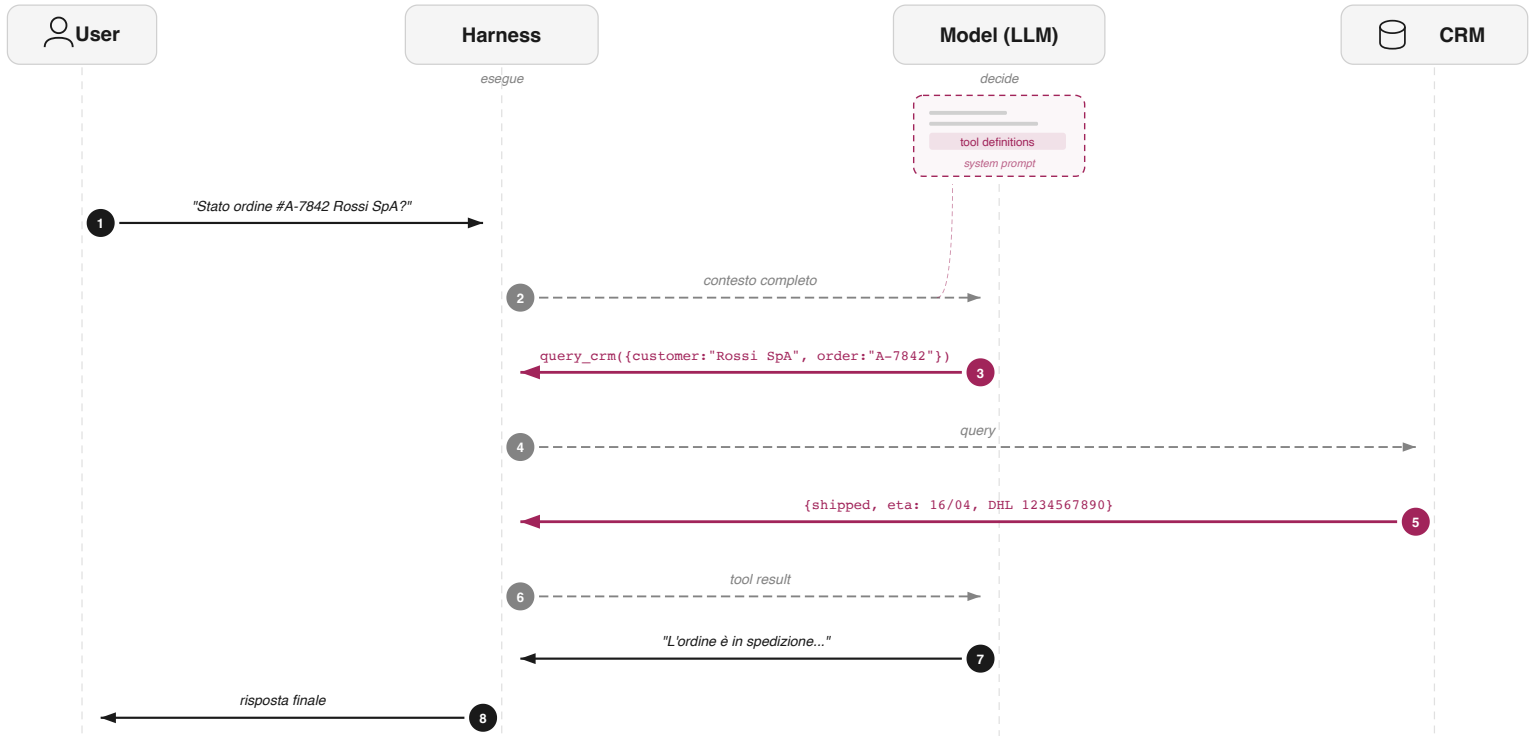
Qual è lo stato dell'ordine **#A-7842** del cliente **Rossi SpA**?

L'ordine #A-7842 (Rossi SpA) è in **spedizione** dal 12 aprile, consegna prevista il **16 aprile**. Tracking DHL `1234567890`.

Una risposta precisa, specifica, verificabile. Eppure questi dati **non esistono nel training del modello** — nessun ordine, nessun gestionale aziendale fa parte di ciò su cui è stato addestrato.

Allora come ha fatto?

La meccanica del tool call



- Il modello non ha eseguito nulla — ha emesso una richiesta strutturata, e l'**harness** ha fatto il lavoro.
- Ogni tool call aggiunge almeno **due round-trip**. Su task complessi, latenza e costo si moltiplicano.
- Il modello sceglie il tool leggendo le **descrizioni nel system prompt**. Descrizione ambigua = scelta sbagliata.

Dall'agente che parla all'agente che agisce

Fino a qui, un modello che genera testo. Da qui, un modello che decide quali azioni intraprendere nel mondo.



> Distinzione fondamentale: il modello non esegue. Emette richieste strutturate di esecuzione. L'esecuzione la fa il software che lo circonda — l'**harness**.

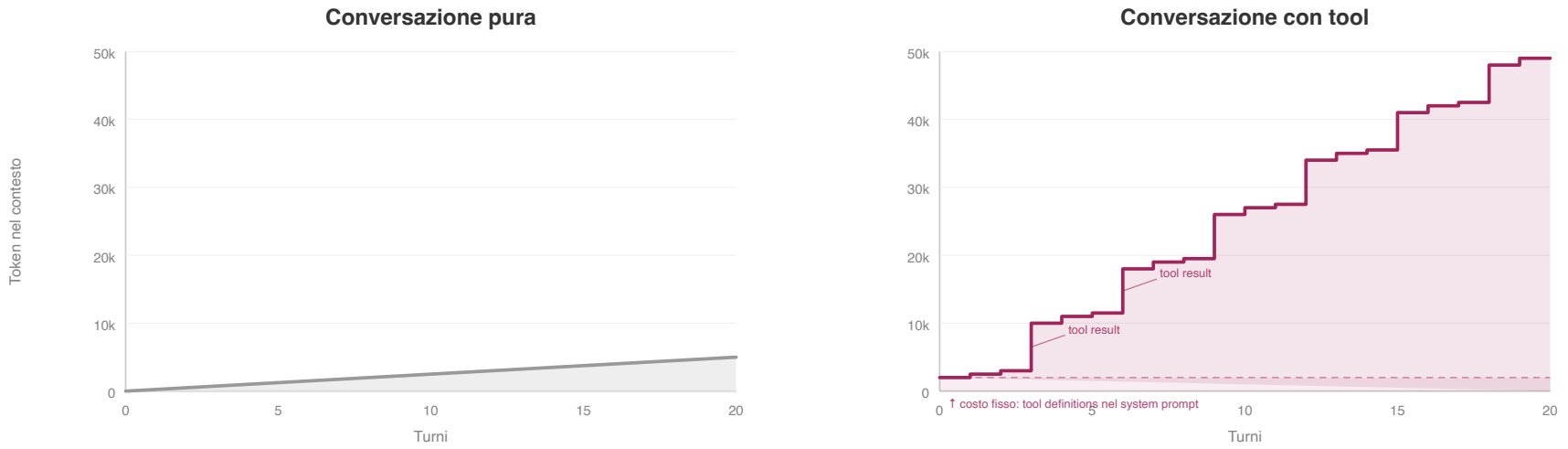
Cosa finisce nel contesto quando si usano tool

Componente	Quando	Dimensione tipica	Persistenza
Definizione tool	Una volta, in cima	100–300 token × nº tool	Tutta la conversazione
Tool call	Ad ogni invocazione	Piccola (nome + argomenti)	Tutta la conversazione
Tool result	Ad ogni invocazione	Variabile — da decine a decine di migliaia di token	Tutta la conversazione
Turni user/assistant	Come prima	Piccola	Tutta la conversazione

🎧 **Esempio: agente customer service con 3 tool (query_crm, search_kb, create_ticket) — ~600 token (def) + ~200 (call) + ~12k (result) = ~13k token di overhead prima della risposta**

Il contesto non contiene solo lo scambio tra utente e agente. Contiene anche tutto ciò che i tool hanno recuperato dal mondo esterno — ogni result, ogni documento restituito, ogni risposta API. E niente viene mai ripulito automaticamente.

I tool accelerano il context rot



> I tool sono la risposta al limite più evidente del modello da solo: non sa quello che non sa, e non può agire nel mondo. Ma ogni tool call deposita nel contesto il suo risultato — e lì resta per tutta la sessione.

- Esporre all'agente solo i tool strettamente necessari. Ogni definizione paga un costo fisso anche se il tool non viene mai chiamato.
- Dimensionare i tool result. Un tool che restituisce 50k token grezzo non è un tool ben progettato — è un carico.
- Preferire tool ben definiti a tool generici. Un tool che fa una cosa sola viene scelto con più precisione.

La prossima tecnica — i sub-agent — è un modo per non pagare questo costo.

Sub-agent: delegare per gestire meglio il contesto

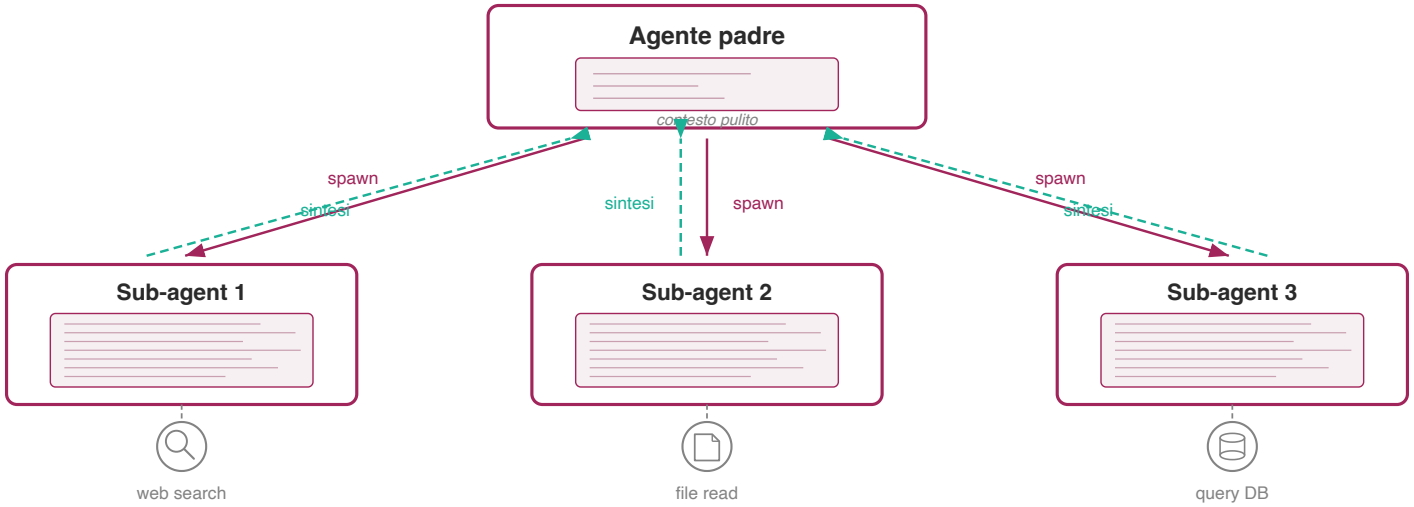
Il problema: ogni tool result rimane nel contesto per tutta la sessione — e satura.

L'analogia: il manager di progetto non legge personalmente i 200 documenti del due diligence. Chiede a un collaboratore di farlo — e di tornare con un executive summary di due pagine. Il manager mantiene il suo contesto strategico pulito. Il collaboratore si sporca le mani.

Il pattern: un agente figlio riceve un task delegato, lavora con il proprio contesto isolato, e restituisce al padre solo il risultato sintetizzato. Il contesto del padre non viene mai contaminato dal processo.



Come funziona il pattern sub-agent



Benefici

- **Risparmio token** — il padre non reitera sul contesto dei figli
- **Isolamento** — padre vede solo il risultato
- **Parallelizzazione** — più figli in contemporanea
- **Specializzazione** — prompt e tool propri
- **Contenimento errori** — fallimento figlio non intacca il padre

Esempi reali

- **Claude Code** — esplorazione codebase
- **Deep Research** — fonti parallele
- **OpenClaw** — agente generalista multi-sub-agent

I sub-agent non sono gratis

Perdita di informazione

Il padre vede solo ciò che il figlio sceglie di riportare. Dettagli scartati nella sintesi sono irrecuperabili.

Debugging complesso

Le tracce diventano alberi, spesso non completamente persistiti. Errori difficili da riprodurre.

Errori composti silenziosi

Un figlio che sbaglia produce una sintesi "ragionevole" che il padre usa come base. L'errore si nasconde negli handoff.

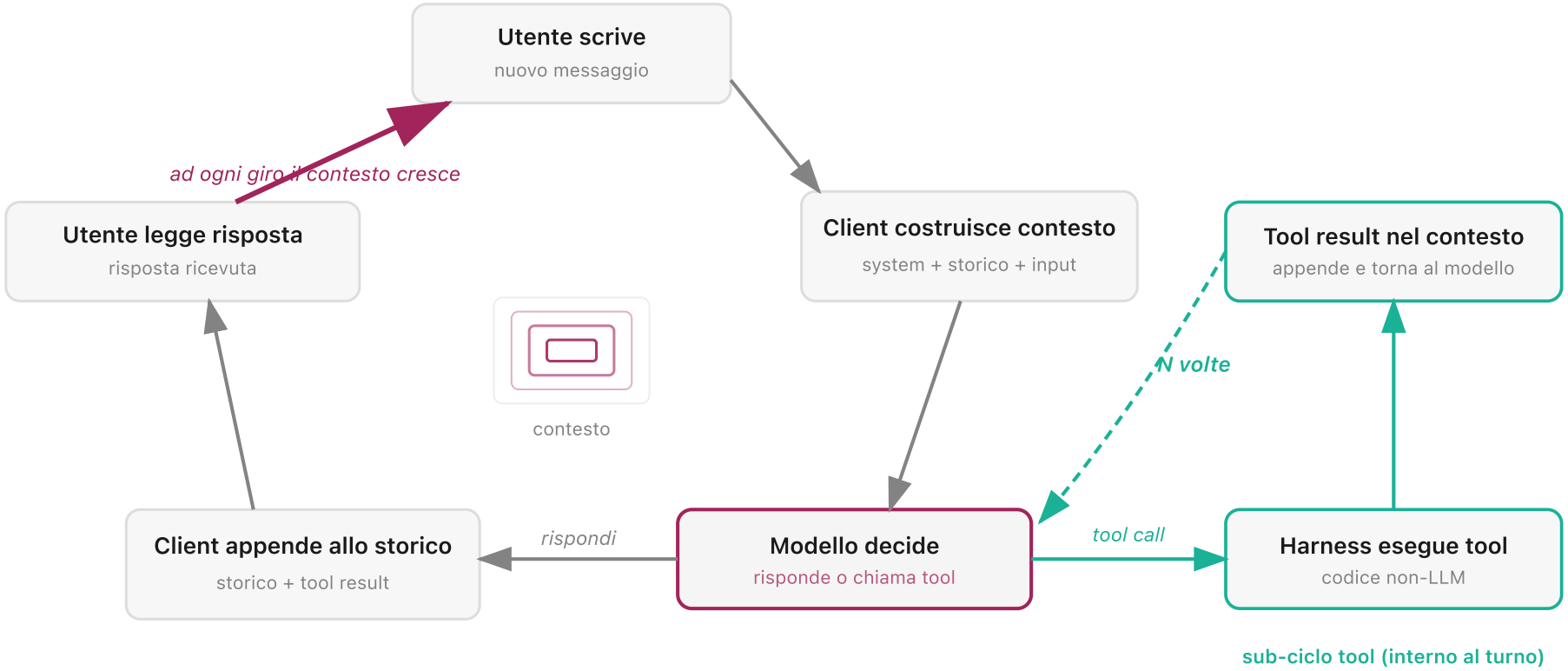
Prompt injection

Un sub-agent con accesso web può essere manipolato da contenuti esterni e contaminare la sintesi.

- Sub-agent vincono su task esplorativi, parallelizzabili, o con contesti specializzati. Perdono dove la tracciabilità è critica.
- Se devi spiegare cosa ha fatto l'agente passo per passo — audit, compliance, debug — il pattern sub-agent rende tutto più difficile.
- Sul costo: misurare, non assumere. A volte un sub-agent costa meno perché il padre non paga il result grezzo.

Loop 3/4: l'agente con tool

Stesso loop conversazionale, con un sub-ciclo tool dentro al turno del modello



Il loop esterno è identico a prima. Cambia solo il nodo **Modello**: invece di rispondere subito, può emettere una tool call, attendere il risultato, e re-iterare N volte prima di chiudere il turno. Il numero di cicli interni non è predeterminato — lo decide il modello a runtime.

> Limiti obbligatori in produzione: un tetto al numero di tool call per turno, un timeout per singola esecuzione, un budget di token per sessione. Senza, l'agente può entrare in cicli infiniti.

Eval: misurare la qualità di un sistema agentico

Suite di test per sistemi agentici. Analoga ai test automatici del software, con tre differenze rilevanti.

Output non binari

Una risposta può essere corretta, parziale, o giusta nel contenuto ma non nel tono. Si usano spesso altri modelli come giudici (LLM-as-judge).

Non deterministici

Lo stesso input non produce sempre lo stesso output. Ogni test si esegue N volte e si osserva una distribuzione.

Dataset da curare

Non si genera automaticamente: serve raccogliere casi reali, edge case ed esempi avversari. È un artefatto che cresce nel tempo.



- Utili per validare modifiche al system prompt, nuovi tool o riconfigurazioni di sub-agent.
- Su progetti agentici la costruzione dell'eval occupa una parte significativa del lavoro.
- Quando si parla di "prompt migliorato", conviene chiedere su quale eval suite è stata misurata la differenza.

Eval: misurare la qualità di un sistema agentico

> Suite di test per sistemi agentici. Analoga ai test automatici del software, con tre differenze rilevanti.

Output non binari

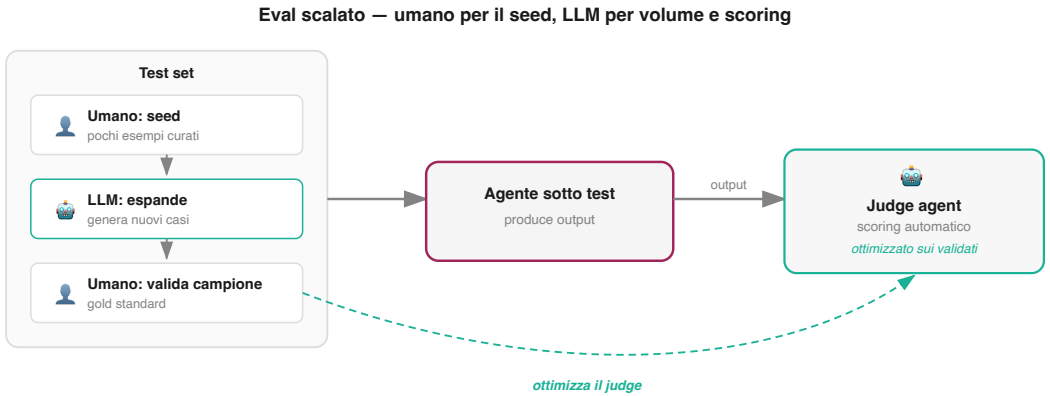
Una risposta può essere corretta, parziale, o giusta nel contenuto ma non nel tono. Si usano spesso altri modelli come giudici (LLM-as-judge).

Non deterministici

Lo stesso input non produce sempre lo stesso output. Ogni test si esegue N volte e si osserva una distribuzione.

Dataset da curare

Non si genera automaticamente: serve raccogliere casi reali, edge case ed esempi avversari. È un artefatto che cresce nel tempo.

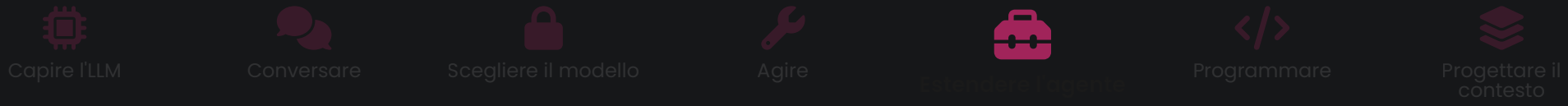


- Utili per validare modifiche al system prompt, nuovi tool o riconfigurazioni di sub-agent.
- Su progetti agentici la costruzione dell'eval occupa una parte significativa del lavoro.
- Quando si parla di "prompt migliorato", conviene chiedere su quale eval suite è stata misurata la differenza.

SEZIONE 5 DI 7

Estendere l'agente

Le cinque famiglie di tool



Tipi di tool — il catalogo

Fino a qui: **la meccanica** del tool call. Da qui: **quali tool** esistono davvero.

5 categorie che coprono la quasi totalità degli agenti oggi in produzione.



API tools



Knowledge access



MCP



Browser use



Code execution

API tools — l'archetipo

Qualunque **endpoint HTTP** diventa una capacità dell'agente definendo lo schema della sua API: nome, descrizione, parametri, autenticazione.

Esempi aziendali

Sistema	Operazione tipica
CRM	Lookup cliente, update record
DB relazionale	Query su dati operativi
Ticketing	Apertura ticket, stato
Email transazionale	Notifiche, comunicazioni
ERP	Ordini, inventario
Pagamenti (Stripe)	Charge, refund, subscription

```
{
  "name": "get_customer",
  "description":
  "Retrieve customer data
  by ID from the CRM.",
  "input_schema": {
    "type": "object",
    "properties": {
      "customer_id": {
        "type": "string",
        "description": "…"
      }
    },
    "required": ["customer_id"]
  }
}
```

name
come l'agente lo chiama

description
su questo testo l'agente decide se usarlo

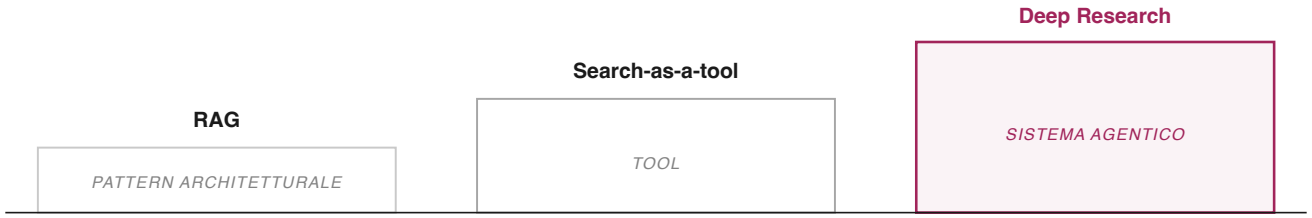
input_schema
parametri tipizzati

PRINCIPIO DI MINIMO PRIVILEGIO

Un tool read-only sul CRM è diverso da un tool che sposta denaro. Esporre all'agente **solo** i tool strettamente necessari al task, con scope il più ristretto possibile.

La famiglia della conoscenza — 3 pattern

Tre modi per accedere a conoscenza non nel training.



PATTERN ARCHITETTURALE

RAG

- Un retriever estrae i chunk rilevanti **prima** della generazione
- I chunk vengono iniettati nel contesto; la risposta è ancorata alla conoscenza proprietaria
- Il modello non decide se cercare — la ricerca avviene sempre

TOOL

Search-as-a-tool

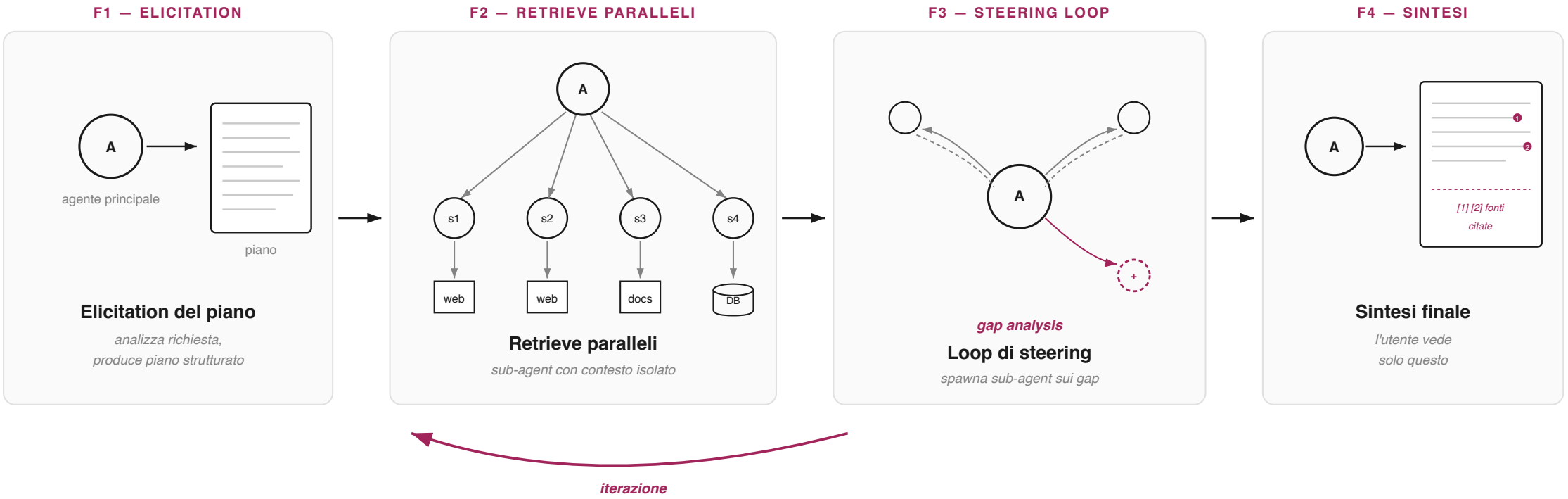
- La ricerca diventa una tool call: il modello decide **quando, cosa, come** cercare
- Abilita l'iterazione: cercare, leggere, raffinare, ricercare
- Risolve il knowledge cutoff in modo dinamico

SISTEMA AGENTICO

Deep Research

- Pianificazione, sub-agent paralleli, loop di steering, sintesi finale con citazioni
- Combina RAG e search orchestrati in multi-fase
- Dettaglio nella prossima slide

Anatomia di Deep Research



F1 — Elicitation del piano · l'agente principale analizza la richiesta e produce un piano strutturato.

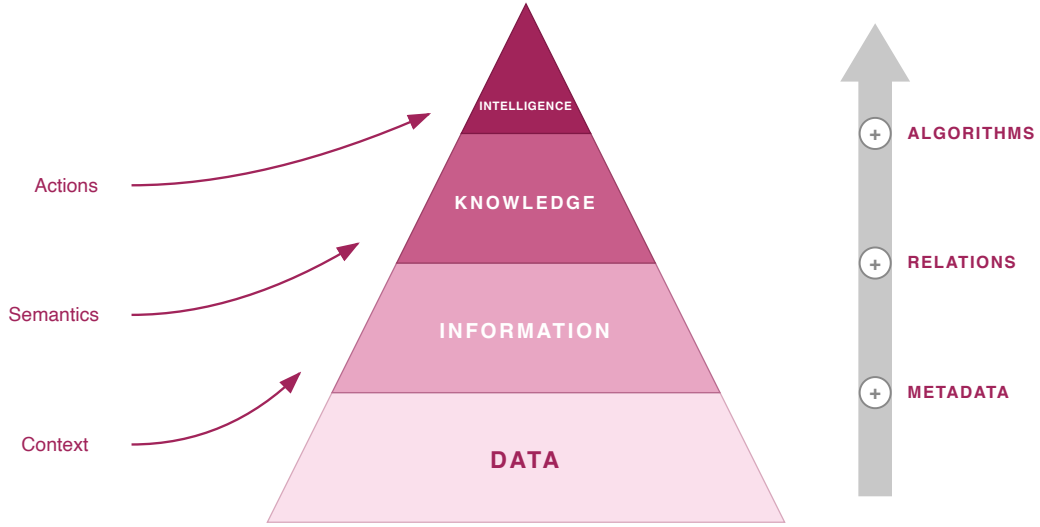
F2 — Retrieve paralleli · sub-agent con contesto isolato esplorano fonti diverse simultaneamente.

F3 — Loop di steering · l'agente principale confronta i risultati col piano e spawna nuovi sub-agent sui gap.

F4 — Sintesi finale · integrazione in documento strutturato con citazioni — l'utente vede solo questo.

Modellazione della conoscenza — 1/3 · dai dati all'architettura informativa

I dati grezzi non bastano. Per essere riutilizzabili — da utenti e da agenti — devono essere arricchiti di **contesto** e **semantica di dominio**: tecnicamente l'informazione c'è, praticamente non è azionabile.

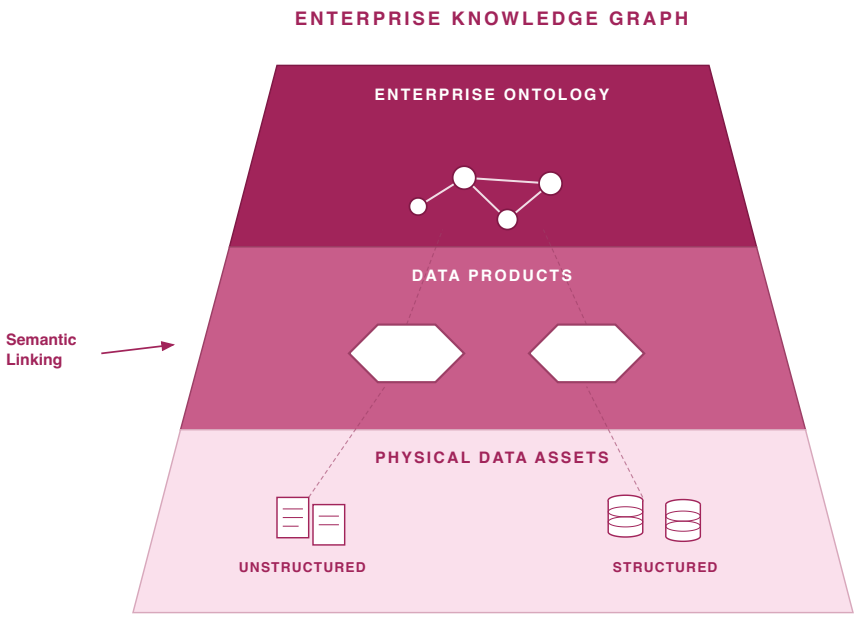


- **Dati** — asset fisici grezzi (tabelle, file, topic, bucket, contenuti non strutturati)
- **+ Metadati → Informazione** — comprensibilità al livello base
- **+ Relazioni → Conoscenza** — semantica di dominio: concetti, attributi, relazioni esplicite
- **+ Algoritmi → Intelligenza** — insight tradotti in decisioni e azioni di business

L'architettura informativa precede logicamente l'adozione dell'AI — anche se in pratica si fanno **in parallelo**, perché l'AI è l'occasione per farla finalmente.

Modellazione della conoscenza — 2/3 · Enterprise Knowledge Graph

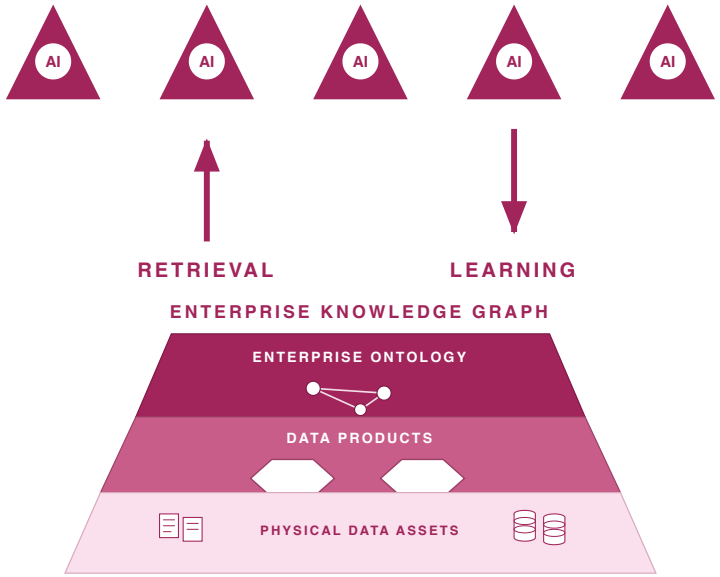
Un **Enterprise Knowledge Graph** virtuale collega la conoscenza semantica di dominio ai dati in piattaforma e ai metadati di contesto: un contesto condiviso per più agenti AI.



- **Ontologia aziendale** — modello a grafo: concetti (Cliente, Prodotto, Ordine), attributi, relazioni (Prodotto "comprato da" Cliente). Più espressiva del Business Glossary.
- **Data Product** — unità di modularizzazione, ownership, deployment. Owner, ciclo di vita, data contract. Registrati nel Data Product Catalog, accessibili via Marketplace.
- **Asset fisici + Semantic Linking** — tabelle, topic, bucket, contenuti non strutturati. I metadati tracciano il collegamento verticale tra ogni asset e i concetti dell'ontologia.

Quattro pilastri: **conoscenza di dominio** come patrimonio differenziante · collegamento esplicito semantica ↔ dati · **interoperabilità** strutturati/non

Modellazione della conoscenza — 3/3 · contesto per gli agenti



L'EKG è il contesto comune che più agenti AI condividono per migliorare ricerca, interrogazione dei dati e supporto agli utenti nell'estrazione di insight.

Retrieval — gli agenti consultano il grafo per recuperare contesto di dominio affidabile e ancorare le risposte a dati reali.

Learning — gli agenti contribuiscono a estendere il grafo, persistendo nuova conoscenza appresa da esperti o da contenuti non strutturati.

Riuso dei dati — Data Product modulari, ricomponibili per più casi d'uso

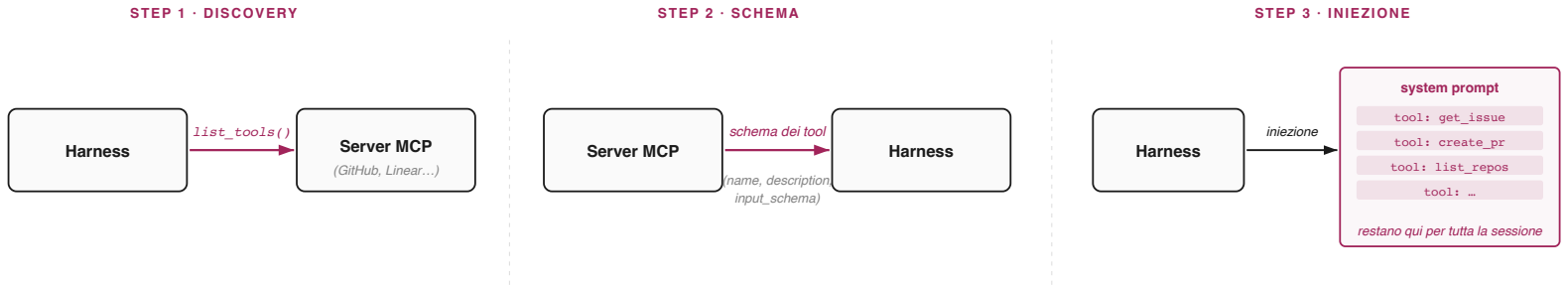
Semantica condivisa — ambiguità ridotte fra unità di business

Gap business ↔ IT — linguaggi avvicinati, cultura dei dati diffusa

L'EKG è la forma aziendale della **skill**: mappa navigabile ai domini di conoscenza, collegata a dati reali e governata. Se il prossimo progetto AI parte con "prendiamo i PDF e mettiamoli in un vector store", **fermarsi** — partire dai domini, dall'ontologia, dai data product che li espongono.

MCP — il protocollo standard per i tool

MCP sta ai tool come **USB** *sta ai dispositivi. Un connettore standard, non un nuovo dispositivo.*



Cos'è (e cosa non è)

- **Non** è un nuovo tipo di tool, né cambia la meccanica del tool call
- **È** un protocollo che standardizza discovery ("quali tool offri?") e invocazione (formato uniforme)
- Dopo il discovery, le tool definition finiscono nel **system prompt** e restano lì per tutta la sessione

Perché conta

- Riduce il vendor lock-in: stessa integrazione su qualunque agente compatibile
- Centinaia di server pubblici: GitHub, Linear, Notion, Gmail, DB — layer degli **app-store per agenti**
- Se un'integrazione AI proprietaria non espone MCP, è vendor lock-in.

⚠ IL TRABOCCHETTO

Ogni server MCP connesso paga token nel contesto: le tool definition sono **sempre** nel system prompt. Connetterne 10 "nel caso servissero" può saturare metà del contesto prima che l'agente inizi a lavorare. Quando un prodotto dichiara "supporta 50 MCP", chiedere quali si attivano di default.

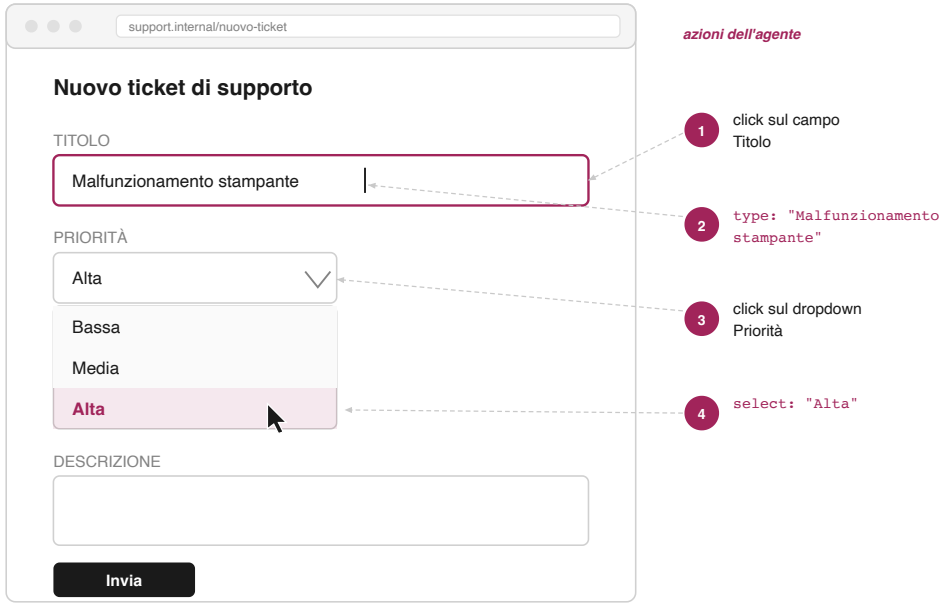
Browser use — l'agente pilota un browser

L'agente riceve **screenshot** della pagina e produce **azioni**: click, scroll, scrittura in form, navigazione.

Claude Computer Use (Anthropic) | Browser Use (open source) | OpenAI Operator

Quando serve	Caveat
SaaS legacy senza API	**Lento** — ogni azione è un round-trip con screenshot
Sistemi interni senza interfaccia programmatica	**Costoso** — gli screenshot come input consumano molti token
Web automation su siti senza endpoint	**Opaco** — azioni viste, non strutturate, difficili da auditare

Scelta di ***ultima istanza****, non di prima — sblocca il long tail delle automazioni aziendali.*



Bash / code execution — il tool universale

L'agente può eseguire comandi shell o codice in un **ambiente sandboxato**. Accesso a filesystem, librerie, script.

UNIVERSALE

Non sblocca una capacità specifica. Sblocca **lo spazio delle capacità**: se c'è bash, c'è accesso a tutto l'universo software.

GENERATIVO

Invece di 20 tool predefiniti, l'agente **scrive uno script** che risolve il task specifico. È il cuore dei coding agent.

COMPONIBILE

Permette di comporre pacchetti di capacità — le **skill**: directory di file, istruzioni e script che l'agente apre ed esegue quando servono. Senza bash, niente skill.

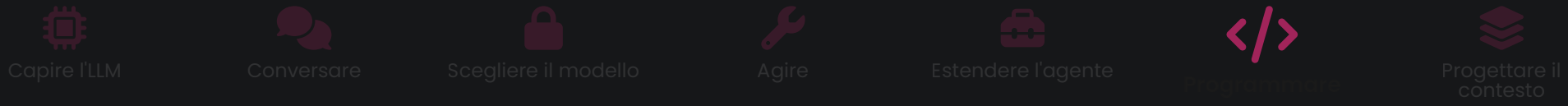
TOOL PIÙ POTENTE = TOOL PIÙ PERICOLOSO

Sempre sandboxato: container isolato, no accesso a reti private, no credenziali sensibili in environment, limiti di risorse espliciti.

SEZIONE 6 DI 7

Programmare

Quando l'agente scrive ed esegue codice



L'agente che scrive ed esegue codice

> Il codice è il **tool di ultima istanza**. Non sblocca una capacità specifica — sblocca **capacità arbitrarie** senza doverle pre-definire.

Due modi di usare il codice in un agente: **come tool interno**, **come prodotto finale**. Condizionano tutto il resto.

Code as tool vs code as product

	Code as tool	Code as product
Cosa fa	Scrive codice per risolvere internamente un task; il codice **non è il deliverable**	Scrive software **come deliverable**: è il risultato del lavoro
Ciclo di vita del codice	**Usa-e-getta**: eseguito una volta, spesso invisibile all'utente, non salvato	**Persistente**: versionato, committato, mantenuto nel tempo

ESEMPIO · CODE AS TOOL

Claude for PowerPoint

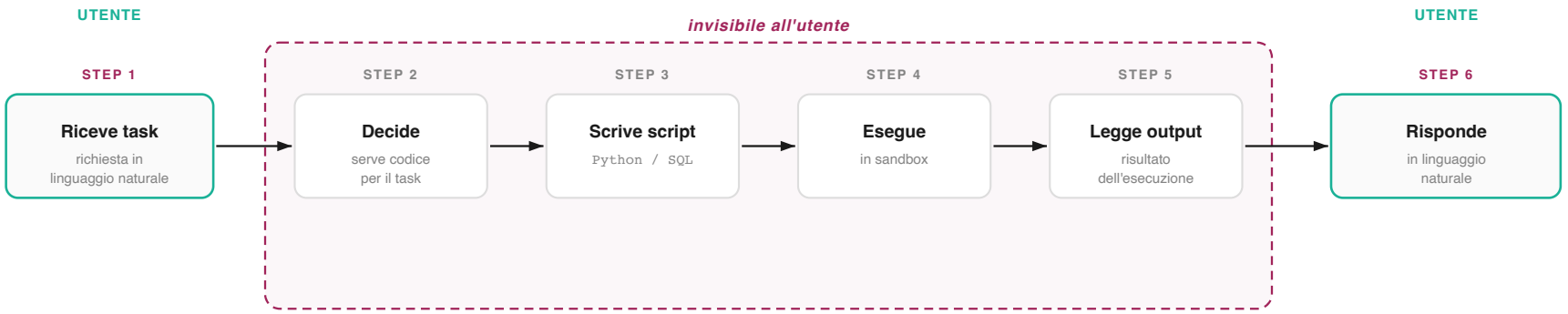
ESEMPIO · CODE AS PRODUCT

Claude Code, Cursor, GitHub Copilot

Quando un fornitore dice "la nostra AI scrive codice", chiedere **quale dei due**. Sono due mondi con beneficiari, rischi e metriche di valore diversi.

Code as tool — il pattern operativo

> **USER** "Qual è stato il fatturato medio per trimestre nel 2023?"



l'agente vede codice ed esecuzione · l'utente vede solo la risposta

L'utente vede la **risposta**, non il codice. Il codice è **plumbing interno**.

Un LLM "a mano" sbaglia la media di 50 numeri. Con **Python** la calcola esatta.

Code as product — coding agent e affidabilità

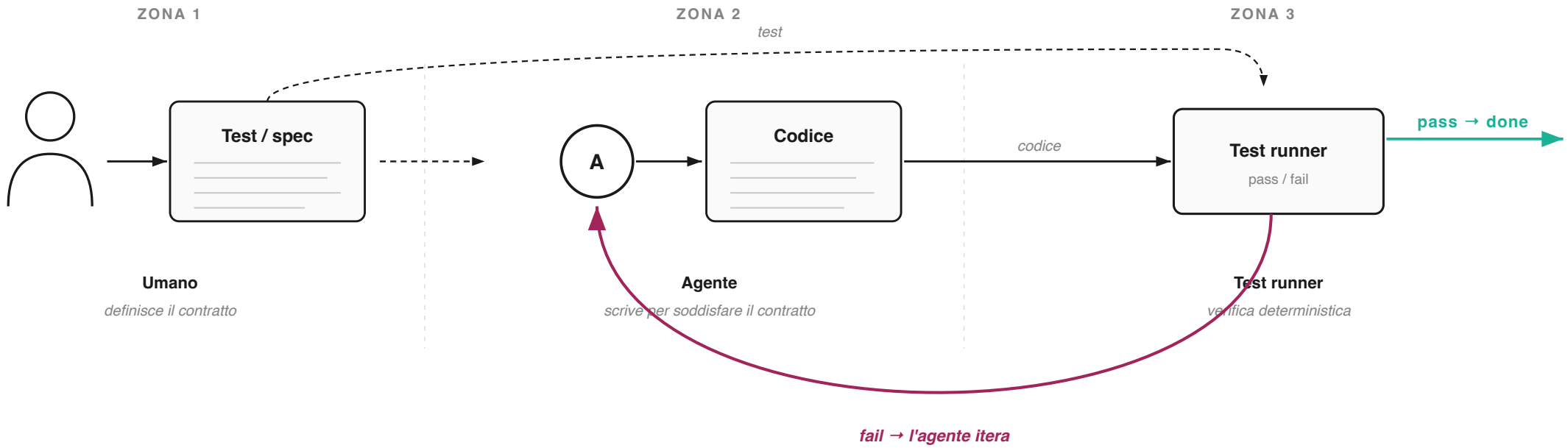
Claude Code | Cursor | GitHub Copilot

SCALA

Stesso pattern del code as tool (scrivi-esegui-verifica), moltiplicato per un progetto reale: **molti file, dipendenze, test suite, git, pull request**. Rientrano in gioco **sub-agent** (esplorare codebase) e **skill** (codificare convenzioni di progetto).

TDD COME MECCANISMO DI AFFIDABILITÀ

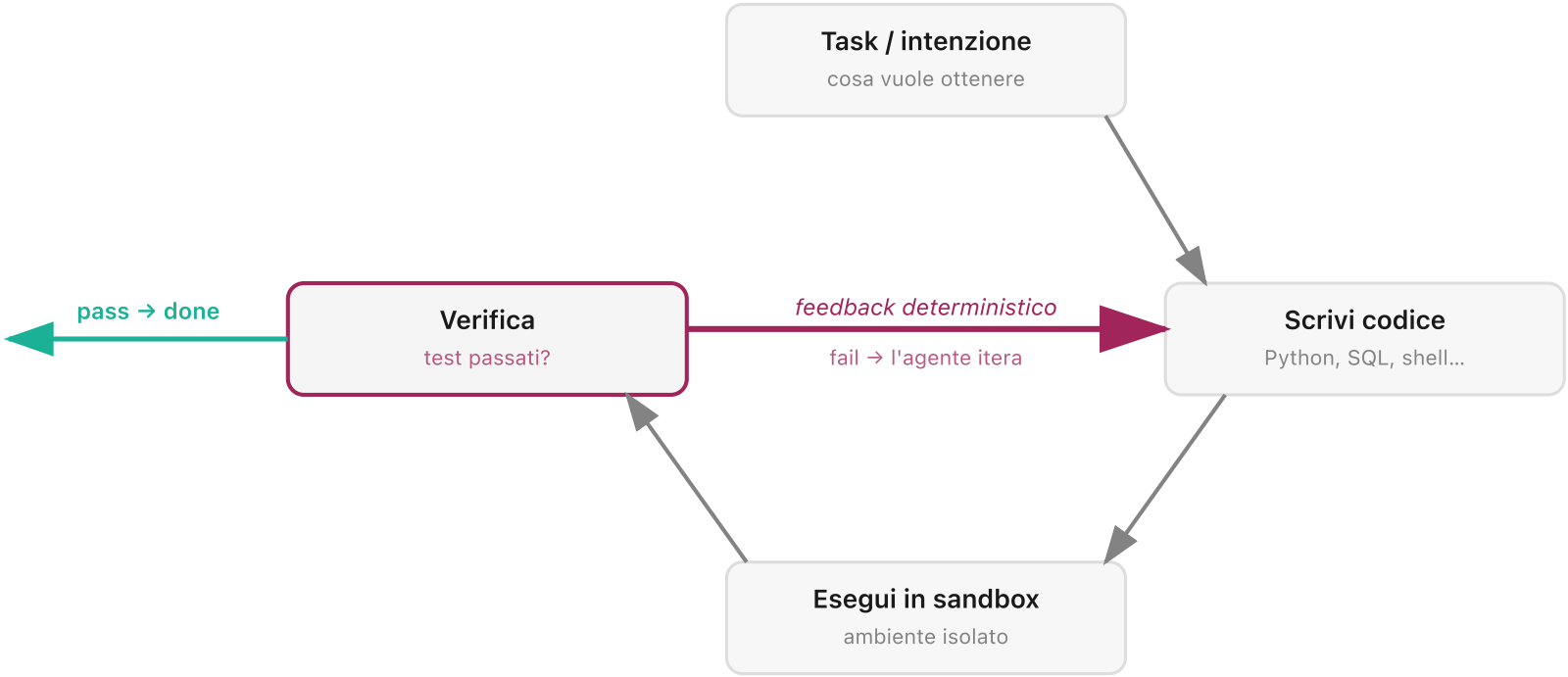
- Si specifica il comportamento atteso come test prima di generare il codice.
- Il test è **contratto**: l'agente sa quando ha finito.
- Il test è **feedback loop deterministico**: l'agente sa se è sulla strada giusta.



Il ruolo umano si sposta su **definizione del contratto** *e* **supervisione**.

Loop 4/4: con verifica deterministica

1° **atomico** (token) · 2° **conversazionale** (turno) · 3° **con tool** (azione) · 4° **con codice** (verifica)



A RUNTIME

L'agente **si corregge da solo**. L'output del test rientra nel contesto come **segnale netto**: più tentativi, convergenza al codice che funziona.

IN TRAINING

Lo stesso loop è il cuore del **RL sul codice**: pass/fail come **reward**. Segnale netto = riaddestramento efficace. Gli LLM migliorano nel codice più rapidamente che in domini con feedback ambiguo.

La verificabilità come predittore di automazione

Abbiamo visto che **nel codice** l'AI migliora rapidamente perché il feedback è netto. Generalizziamo.

> Se una professione intellettuale ha **output immediatamente e binariamente verificabile**, è a **pressione di automazione elevata**. Se il successo richiede **giudizio qualitativo, negoziazione, contesto sociale**, la pressione è più bassa — almeno per ora.

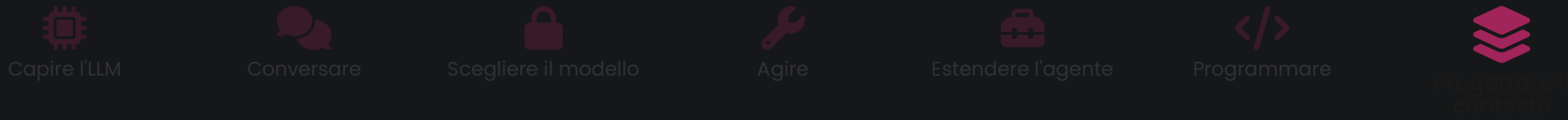
Alta verificabilità → pressione alta	Bassa verificabilità → pressione bassa
Scrittura di codice con test	Strategia aziendale
Debugging	Negoziazione
Ottimizzazione logistica (routing, scheduling)	Leadership
Dimostrazione matematica in sistema formale	Vendita consulenziale complessa
Calcolo strutturale e simulazione fisica	Diagnosi medica in casi ambigui
Riconciliazione contabile	Decisioni etiche

Caveat. *Verificabilità non è sostituibilità totale. Resta il ruolo umano di* **definire il contratto**, **gestire casi fuori pattern**, **rispondere delle conseguenze**. *Il pattern sposta il lavoro, non sempre lo cancella.*

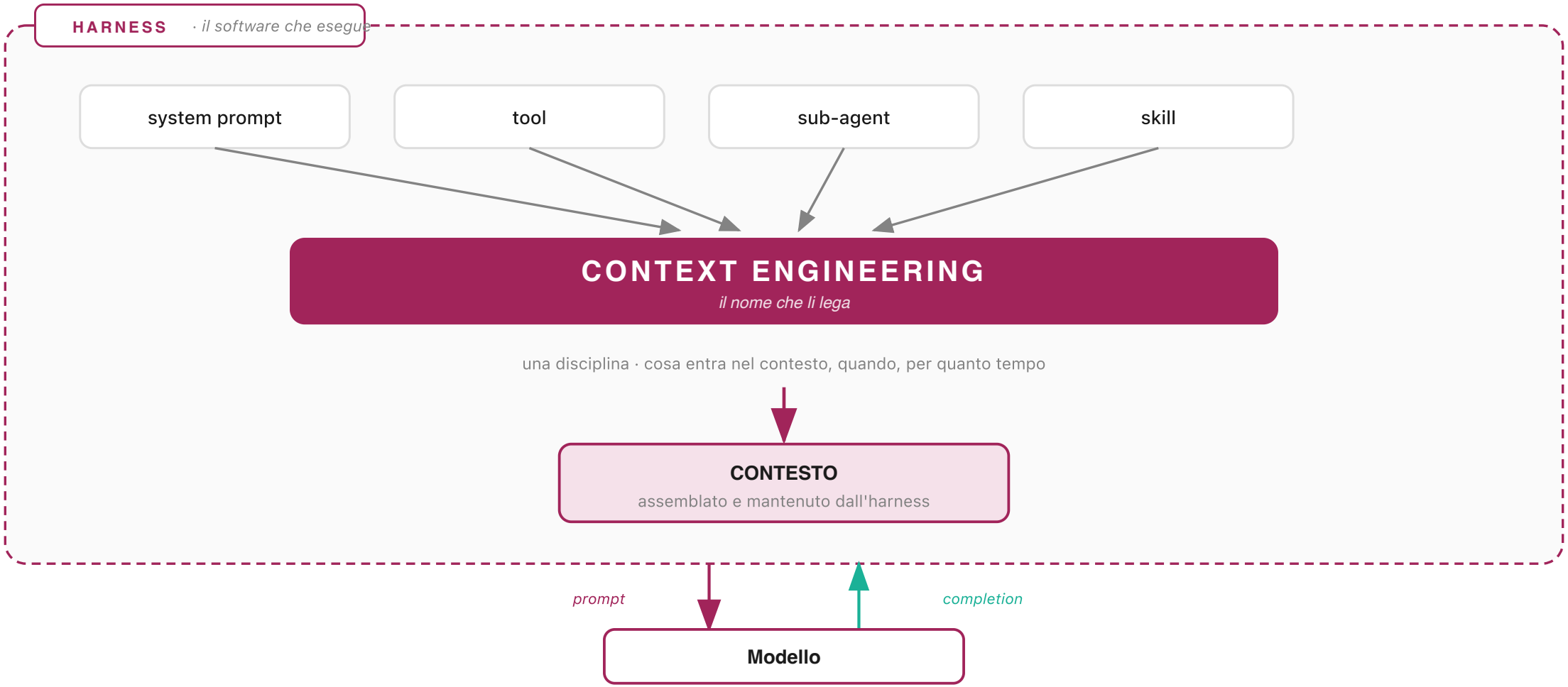
SEZIONE 7 DI 7

Progettare il contesto

Ciò che circonda il modello



Context engineering



Harness — chi fa davvero le cose

Il **software che circonda il modello** e gestisce tutto ciò che non è pura generazione di testo.

- **Costruisce e mantiene il contesto** — assembla system prompt, storico turni, tool definition, tool result a ogni chiamata API
- **Esegue i tool quando il modello li richiama** — intercetta la richiesta strutturata e la traduce in chiamata reale
- **Applica policy di sicurezza e autorizzazione** — decide quali azioni sono permesse in base al contesto e all'utente
- **Orchestra i sub-agent** — lancia agenti figli, raccoglie le sintesi, le restituisce al padre
- **Gestisce lo stato persistente** — memoria, sessioni, contesti tra conversazioni diverse
- **Fornisce osservabilità** — log, tracce, metriche per debugging e monitoring in produzione

Quando si dice "Claude ha fatto X", Claude ha **deciso** X e l'harness ha **eseguito** X. Il confine è sottile ma cruciale — quasi tutta la sicurezza, l'affidabilità, la qualità di un agente in produzione vivono nell'harness.

Context engineering come disciplina

Progettare **cosa vive nel contesto** del modello a ogni chiamata — includere ciò che serve, escludere ciò che non serve, decidere per quanto resta. È il lavoro principale dell'harness.

Compaction e pruning

Ridurre il contesto che cresce.

Progressive disclosure

Caricare on-demand solo ciò che serve.

Memoria

Persistenza oltre la singola sessione.

Sandbox

Confini dell'esecuzione — adiacente: non è contesto, ma condivide lo stesso attore.

Una leva per slide, nei prossimi blocchi.

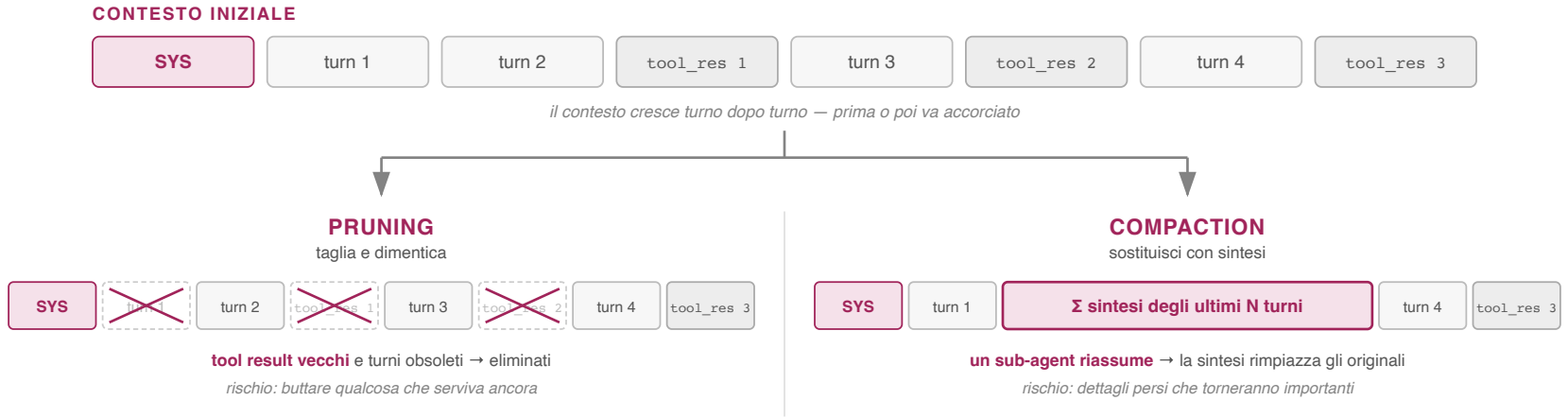
Compaction e pruning — gestire il contesto che cresce

Pruning

- **Definizione** — tagliare parti del contesto non più rilevanti (tool result vecchi, turni obsoleti)
- **Policy** — quali informazioni sono sicure da dimenticare?
- **Rischio** — dimenticare qualcosa che serviva

Compaction

- **Definizione** — sostituire parti del contesto con una loro sintesi
- **Policy** — tipicamente un sub-agent riassume gli ultimi N turni e la sintesi rimpiazza gli originali
- **Rischio** — la sintesi perde dettagli che emergeranno come importanti più avanti

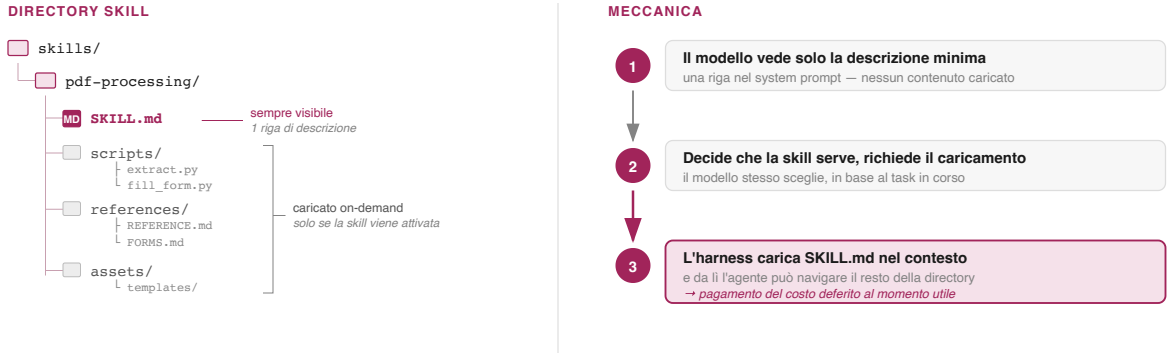


Entrambe sono **lossy**: si perde informazione in cambio di spazio. La scelta di cosa comprimere o buttare è una delle decisioni più delicate di un agente in produzione. **Non esiste policy universale** — dipende dal task.

Progressive disclosure — il pattern skill

	MCP	Skill
Tool definition	**Sempre** in contesto	**Descrizione minima** in contesto
Costo	Fisso all'inizio della sessione	Pagato **solo se** la skill viene caricata
Efficienza	Inefficiente se pochi tool servono davvero	Carica **solo il necessario**

Skill: una **directory** con documentazione, istruzioni, esempi, eventuali script. `SKILL.md` spiega al modello cosa offre e come usarla.



ESEMPIO

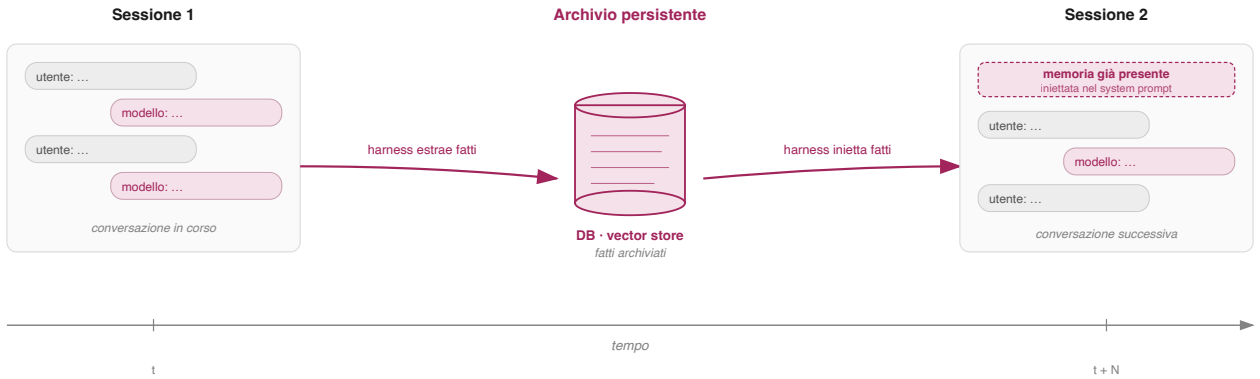
Skill pdf-processing: `SKILL.md` + `scripts/` (codice), `references/` (doc), `assets/` (template).

MECCANICA

In Claude Code il meccanismo è il tool `bash`: il modello esplora la directory e legge i file.

Memoria — demistificazione e meccanica

"Memoria" è un termine abusato. Suggerisce che il modello **ricordi** *nel senso umano. Non è così.*



Il modello è stateless — la "memoria" vive interamente nell'harness e nell'archivio.

- Il modello è **stateless** — nessuna memoria tra chiamate (richiamo slide 10)
- La "memoria" è un **sistema esterno** gestito dall'harness
- L'harness decide **cosa salvare** fuori dal contesto corrente (fatti sull'utente, preferenze, sintesi)
- Lo salva in **archivio persistente** (DB, vector store, file) e al turno successivo **recupera i fatti rilevanti** e li inietta nel contesto

> Memoria dell'agente = **RAG sulle sue stesse conversazioni passate**. Non è che il modello "si ricorda di voi" — è che l'harness ha archiviato fatti su di voi e ve li re-inietta nel contesto quando parlate di nuovo.

Memoria — fasi operative e rischi

3 fasi operative

- **Estrazione** — un sub-agent scorre la conversazione ed estrae fatti memorizzabili ("l'utente preferisce risposte brevi")
- **Gestione** — i fatti vengono organizzati, deduplicati, aggiornati quando cambiano
- **Recupero** — al turno successivo, i fatti rilevanti vengono iniettati nel contesto

4 rischi

- **Memoria obsoleta** — salva un fatto che cambia nel tempo, non si aggiorna da sé
- **Privacy** — cosa viene salvato, per quanto, con quale consenso (GDPR)
- **Cross-contamination** — memoria di una conversazione influenza un'altra dove non dovrebbe
- **Gaslight da memoria** — l'agente "ricorda" cose false iniettate in passato da utente o attaccante

Sandboxes — confini espliciti per l'esecuzione

Ambiente isolato in cui l'agente esegue codice, bash, tool che manipolano il mondo — con **confini tecnici verificabili**.

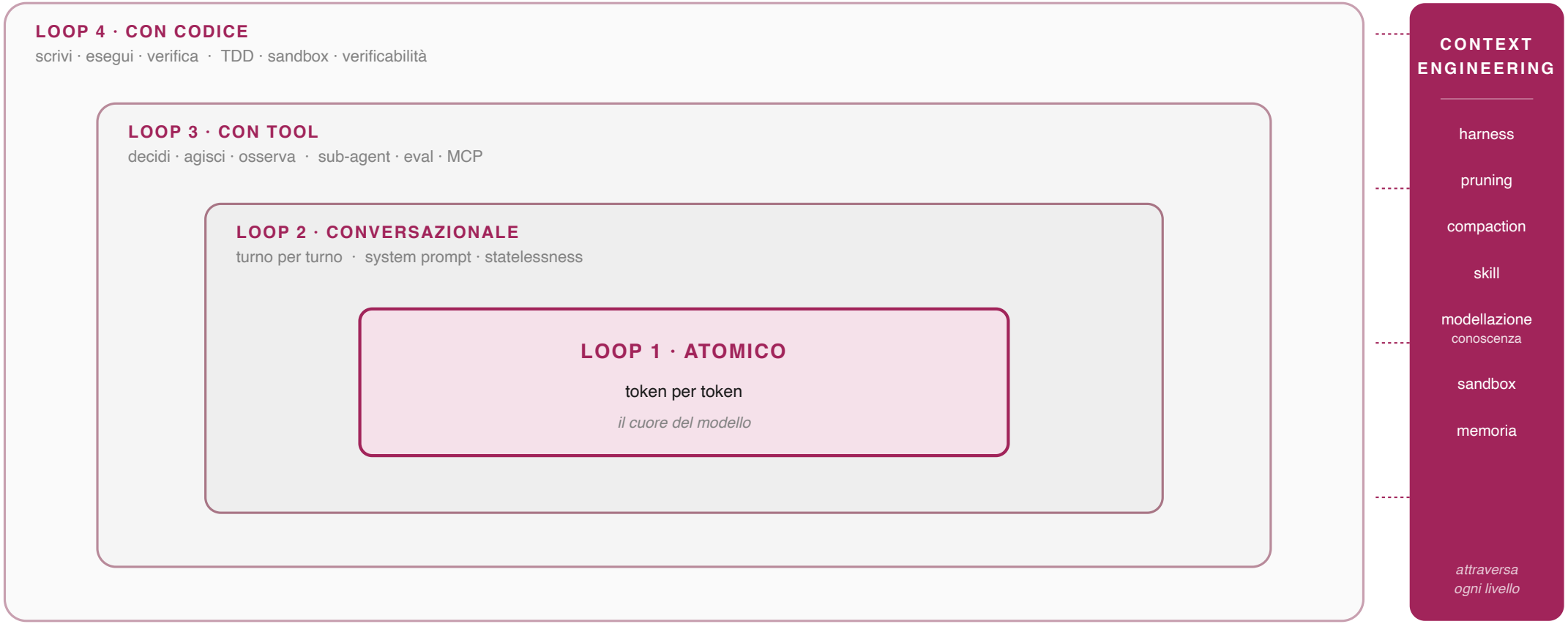
Confine	Perché conta
Filesystem limitato	L'agente accede solo alle cartelle di lavoro designate
Rete limitata	Solo endpoint autorizzati — no exfiltration, no lateral movement
Credenziali limitate	Minimo privilegio — l'agente non ha più accesso di quanto serve
Tempo limitato	Ogni esecuzione scade — previene loop infiniti e consumo silenzioso
Risorse limitate	CPU, memoria, disco espliciti — previene denial-of-service accidentale

Sandbox è alla sicurezza degli agenti quello che il **container** è al deployment del software. **Nessun agente serio gira senza.**

Harness landscape — chi c'è oggi

Il modello è sempre più **commodity**. *La differenziazione si sposta qui.*

Building block e composizione



Quattro loop impilati. Una disciplina che li attraversa tutti.

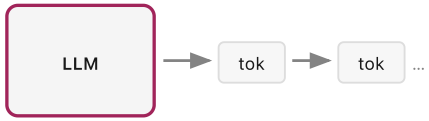
Da domani, quando si parlerà di AI agentica, si sentiranno suonare dei concetti. **Quelli sono i building block.**

I quattro loop in un colpo d'occhio

I quattro loop in un colpo d'occhio

1/4 · ATOMICO

token per token



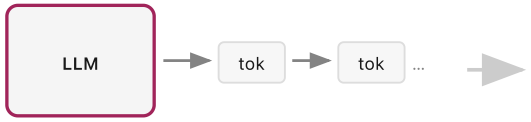
una distribuzione di probabilità
su ogni prossimo token

genera un token alla volta

I quattro loop in un colpo d'occhio

1/4 · ATOMICO

token per token

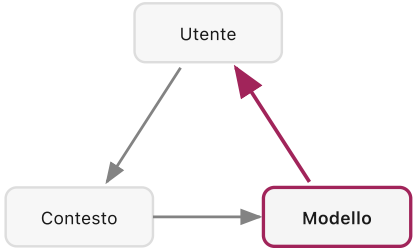


una distribuzione di probabilità
su ogni prossimo token

genera un token alla volta

2/4 · CONVERSAZIONALE

turno per turno



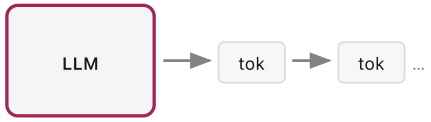
il contesto cresce ad ogni giro

statelessness + system prompt

I quattro loop in un colpo d'occhio

1/4 · ATOMICO

token per token

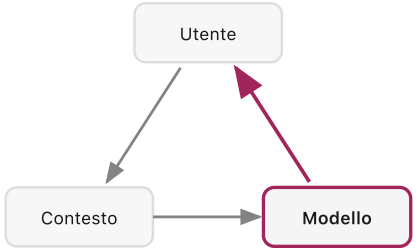


una distribuzione di probabilità
su ogni prossimo token

genera un token alla volta

2/4 · CONVERSAZIONALE

turno per turno

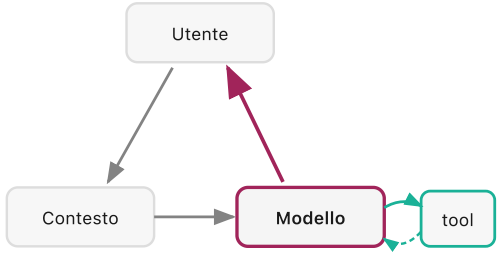


il contesto cresce ad ogni giro

statelessness + system prompt

3/4 · CON TOOL

decidi, agisci, osserva



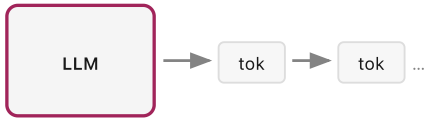
N volte

il modello decide cosa fare

I quattro loop in un colpo d'occhio

1/4 · ATOMICO

token per token

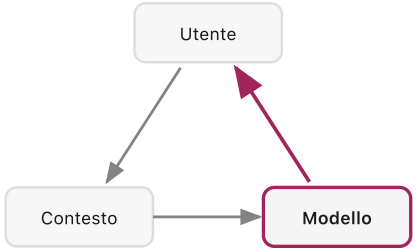


una distribuzione di probabilità
su ogni prossimo token

genera un token alla volta

2/4 · CONVERSAZIONALE

turno per turno

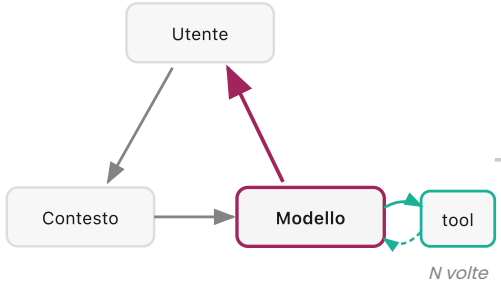


il contesto cresce ad ogni giro

statelessness + system prompt

3/4 · CON TOOL

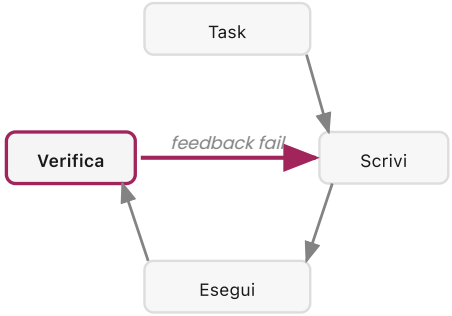
decidi, agisci, osserva



il modello decide cosa fare

4/4 · CON CODICE

scrivi, esegui, verifica



si corregge da solo